

Foundation of Cybersecurity

AA 2023 - 2024

Saverio Mosti

Sommario

Leggimi.....	5
Cifrari perfetti.....	5
Definizione di cifrario perfetto.....	5
Teorema di Shannon.....	5
OTP - One Time Pad.....	6
OTP Pratico - Stream Cipher.....	6
Definizione di sicurezza computazionale.....	7
Random Bit Generators.....	8
TRBG - True Random Bit Generators.....	8
Hardware-based generators.....	8
Sequenze con bias.....	9
De-skewing.....	9
Sequenze correlate.....	9
Software-based generators.....	9
PRBG - Pseudo-Random Bit Generators.....	10
Sicurezza del PRBG.....	10
Sicurezza computazionale del PRBG.....	11
Test del “prossimo bit” (forward unpredictability).....	11
Backward unpredictability.....	11
Periodicità e grado di un PRBG.....	11
LCG - Linear Congruential Generator.....	11
LFSR - Linear Feedback Shift Register.....	12
Content Scrambling System.....	12
CSPRNG - Cryptographically Secure PRBG.....	12
Cifratura simmetrica e cifrari a blocchi.....	12
Cifrario a blocchi perfetto (true random cipher).....	13
Dictionary attack.....	13
DES - Data Encryption Standard.....	13
Feistel Network.....	14
2DES.....	16
Meet-In-The-Middle.....	16
3DES.....	17
Perchè non si va oltre a 3DES?.....	18
AES - Advanced Encryption Standard.....	18
Encryption Mode.....	19
CBC - Cipher Block Chaining.....	19
CTR - Counter Mode Encryption.....	20
ECB - Electronic CodeBook.....	20
Crittografia asimmetrica.....	21

Th - Un cifrario asimmetrico NON può essere perfetto.....	21
Small Plaintext Attack.....	21
RSA.....	22
Generazione delle chiavi di RSA.....	22
Dimostrazione della correttezza di RSA.....	22
Performance RSA.....	23
Calcolo di e	23
Calcolo di p e q	23
Operazioni in modulo.....	24
Square-and-multiply.....	24
Plain-RSA.....	25
Plain-RSA è malleabile.....	25
Adaptive-Chosen Ciphertext Attack.....	26
Plain-RSA è deterministico.....	26
Small Plaintext Attack.....	26
Common Modulus Attack.....	26
RSAP - RSA Problem (DLP).....	27
Diffie-Hellman Key Exchange (DHKE).....	28
Proprietà di Z_p	28
Ciclicità.....	29
Teorema 8.2.3 - Fermat Little Theorem.....	30
Teorema 8.2.4.....	30
Teorema 8.2.5 - Cyclic Subgroup Theorem.....	30
Teorema 8.2.6 - Teorema di Lagrange.....	30
Teorema 8.2.7 - Cyclic Subgroup Theorem.....	30
Sicurezza Diffie-Hellman.....	31
Vantaggi tratti dai gruppi ciclici.....	31
MiM in DHKE.....	32
Firma Digitale.....	32
Secret Splitting.....	33
Bit commitment.....	33
Double spending.....	33
Certificati digitali.....	35
Rilascio di un certificato (CA-generated keys).....	35
Backup della chiave privata.....	36
Threshold Crypto.....	36
Verifica di un certificato.....	36
CRL - Certificate Revocation List.....	37
OCSP - Online Certificate Status Protocol.....	37
Trust Model.....	37
Centralizzato.....	37

Non centralizzato.....	37
Cross Verification.....	38
Browser Verification.....	38
Funzioni di hash.....	38
Le proprietà di una funzione Hash.....	38
Compressione.....	38
Pre-image resistance.....	39
2° Pre-image resistance.....	39
Strong collision resistance.....	39
Blackbox attacks.....	39
Guessing attack.....	39
Birthday attack.....	39
Schema Merkle-Damgaard.....	40
Come realizzare le $h()$	40
Hash con cifrari a blocchi.....	41
Schema di Davies-Meyer.....	41
Merkle Tree.....	41
MAC.....	43
Attacchi al MAC.....	43
Chosen-message attack.....	44
Forgery Selective.....	44
Forgery Existential.....	44
HMAC.....	44
HMAC per autenticazione e confidenzialità.....	44
Attacchi contro HMAC.....	44
Attacco al prefisso.....	44
Timing Attack.....	45
Schema HMAC 1.....	45
CBC-MAC.....	45
CMAC.....	46
GMAC.....	46
ECC - Curve Ellittiche.....	47
Addizione.....	47
Neutrale.....	48
Inverso.....	48
Sottogruppi Ciclici.....	48
Th Hasse - Stima del valore di $\#E$	48
ECDLP - Logaritmo Discreto su Curve Ellittiche.....	50
ECDHKE - DHKE con curve ellittiche.....	51
Attacchi al ECDHKE.....	51
Session key establishment.....	52

One-pass Key Transport.....	52
Challenge-Response Key Transport.....	52
Key Agreement.....	53
Key distribution problem.....	53
Key Distribution Center.....	53
Perfect Forward Secrecy.....	55
PKE-based Key Exchange.....	55
Ephemeral Diffie-Hellman (PSK-DKE).....	55
RSAE - Ephemeral RSA.....	56
Direct Authentication.....	56
Station-to-Station Protocol.....	56
Kerberos.....	58
Protocollo senza delegazione.....	59
Protocollo con delegazione.....	61
Protocollo Needham-Schroeder.....	62
Altro.....	62
One-Time Passwords.....	62
Password storage.....	63
Rainbow Attack.....	63
Bit di sicurezza.....	63
Esercizi.....	64
Esercizio su CTR e CBC.....	64
Esercizio sicurezza protocolli 1.....	65
Esercizio session key.....	67
Esercizio identification protocol 1.....	68
Esercizio sicurezza di una hash function 1.....	69
Esercizio sulla sicurezza di uno stream cipher 1.....	70
Esercizio identification protocol 2.....	71
Esercizio sulla sicurezza di una firma 1.....	72
Domande orale.....	73

Leggimi

Questo documento non contiene appunti presi in classe, ma anzi una raccolta di definizioni, teoremi, algoritmi e concetti; senza tenere conto dell'ordine con cui sono stati spiegati in classe.

Non tutto ciò che è stato spiegato in classe è contenuto in questo documento, ma solo gli argomenti più chiesti all'esame e/o su cui il professore ha messo enfasi durante le lezioni.

Questo documento non contiene nulla riguardo la parte pratica del corso, che invece è spiegata nei laboratori.

Consiglio di usare questo documento durante la preparazione all'esame scritto e orale, oppure come linea guida degli argomenti trattati e non come sostituto di appunti presi in classe.

Tutti gli argomenti che sono stati oggetto di almeno una domanda all'esame scritto o all'esame orale sono presenti in questo documento (*nel periodo dal 2022 al 2024*), ovviamente contiene anche qualche cosa in più.

In fondo, sono presenti alcuni esercizi che sono usciti negli esami scritti dal 2022 al 2023.

Cifrari perfetti

Definizione di cifrario perfetto

$$\forall m, m' \in M, \forall c \in C, P_r[E_k(m) = c] = P_r[E_k(m') = c]$$

- Dati due messaggi possibili (*quindi appartenenti allo spazio dei messaggi M*).
- Dato un crittogramma possibile (*quindi appartenente allo spazio dei crittogrammi C*).

La probabilità che il crittogramma c sia ottenuto cifrando m o m' è la stessa. In parole povere: c non dà informazioni riguardo a m .

Teorema di Shannon

In un cifrario perfetto $|K| \geq |M|$.

1. Supponiamo per assurdo che ci sia un cifrario perfetto con $|K| < |M|$
2. Comunque, $|C| \geq |M|$ altrimenti il cifrario non sarebbe invertibile, poiché ad un crittogramma potrebbero corrispondere due o più messaggi.

3. Inoltre di sicuro $|K| < |C|$
4. Sia $m \in M$ tale che $P_r[M = m] \neq 0$ (un messaggio che può essere inviato nel canale).
5. Sia $c_i = E_{k_i}(m)$ con $k_i \in K$ (tutti i crittogrammi ottenibili cifrando m).
6. Dato il punto (3), $\exists c \in C, c \neq c_i \forall i$
7. Quindi $P_r[M = m | C = c] = 0 \neq P_r[M = m]$

Il punto (7) contraddice l'ipotesi di perfezione del cifrario: poiché c'è almeno un crittogramma c che non può essere ottenuto da m e quindi, vedendo c , posso escludere che è stato inviato m .

Quindi in un cifrario perfetto:

- La lunghezza in bit della chiave è maggiore o uguale alla lunghezza in bit del messaggio.
- Chiave composta da bit randomici.

OTP - One Time Pad

One-time pad è per definizione un cifrario perfetto:

$$c = m \oplus k$$

$$m = c \oplus k$$

Con c, m, k a n bit.

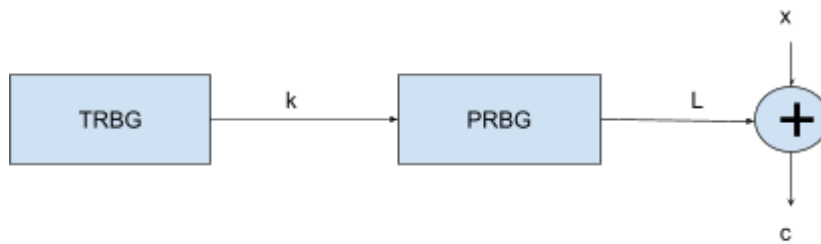
Se k è composta da n bit perfettamente casuali, allora c è una sequenza di bit perfettamente casuali.

OTP Pratico - Stream Cipher

Il problema di OTP è la lunghezza della chiave.

Posso generare la chiave partendo da una sequenza randomica molto più corta, senza perdere la perfezione del cifrario?

Sì se la chiave è generata con un generatore puramente randomico.



Definizione di sicurezza computazionale

Quando (nella teoria e nella pratica) un cifrario si può dire computazionalmente sicuro?

Un cifrario è computazionalmente sicuro se il livello di computazione richiesto per romperlo (*usando l'attacco più efficace noto al momento*) eccede di un certo margine le risorse computazionali dell'avversario ipotizzato.

In pratica, un sistema è computazionalmente sicuro quando ha almeno 80 bit di sicurezza.

Random Bit Generators

RNG: proprietà per la sicurezza?

In cifrari come OTP, la chiave k deve essere composta da n bit casuali, altrimenti viene meno la perfezione.

Più i bit della chiave sono incorrelati tra loro e più la chiave è efficace. Ma non sempre è possibile avere una cosa del genere: ottenere uno stream di n bit casuali può anche essere impossibile.

TRBG - True Random Bit Generators

L'output del generatore è puramente casuale e non può essere né predetto o riprodotto da un avversario.

Data una sequenza di n bit (*supponendo che 0 e 1 hanno la stessa probabilità di uscire*) uscita da un generatore di questo tipo, la probabilità di “*azzeccare*” questa sequenza in modo casuale da parte di un avversario è $\frac{1}{2^n}$

Hardware-based generators

Sono basati spesso sul campionamento di fenomeni fisici come ad esempio:

- Movimento termico all'interno della CPU. (*Intel*)
- Desincronizzazione di un clock
- Decadimento radioattivo.
- Movimento all'interno di lampade lava. (*Cloudflare*)
- Audio rilevato dal microfono della webcam.

Esiste un dispositivo hardware che campiona questi fenomeni e produce una sequenza di bit in uscita.

Pro del hardware-based

- Sono i più efficaci, la sequenza di uscita è puramente randomica.

Contro del hardware-based

- Più costosi e complessi da creare.
- Soggetti all'influenza esterna.
- Soggetti a malfunzionamenti → Richiedono test periodici.
- Vulnerabili all'osservazione.
- Vulnerabili alla manipolazione → Richiedono test periodici.

- I bit prodotti hanno spesso bias o sono correlati.

Sequenze con bias

$p = P(1) \neq P(0) = 1 - p$, ovvero la probabilità di emettere 1 o 0 non sono perfettamente uguali.

Ovviamente p è un parametro sconosciuto, ma può essere stimato. Si risolve con il de-skewing.

De-skewing

Tecnica per rimuovere il bias da una sequenza di bit randomica.

Dato un generatore che produce bit con bias ma non correlati, si raggruppano i bit in uscita dal generatore in coppie e si produce un solo bit:

- 00 → **Scartato**.
- 01 → 1
- 10 → 0
- 11 → **Scartato**.

Il risultato è una sequenza senza bias e non correlata, ma di lunghezza dimezzata.

Sequenze correlate

$$P(b_n = 1 | b_{n-1} = 0) \neq P(b_n = 1 | b_{n-1} = 1)$$

$$P(b_n = 0 | b_{n-1} = 0) \neq P(b_n = 0 | b_{n-1} = 1)$$

Ovvero la probabilità di emettere 1 o 0 dipende anche dai bit emessi prima.

Software-based generators

Si basano su fonti di randomicità che non dipendono da fenomeni fisici come ad esempio:

- Clock di sistema.
- Contenuto dei buffer di input e output.
- Valori del sistema operativo.
- Periodo di tempo tra due interrupt generici.

L'efficacia dipende dal numero di fonti di randomicità usate contemporaneamente: più ce ne sono e più è efficace (*spesso si usano anche funzioni di hash sicure per produrre il risultato finale*).

Pro del hardware-based

- Economici.
- Non soggetti all'influenza esterna (*alla fine sono dei normali programmi*).

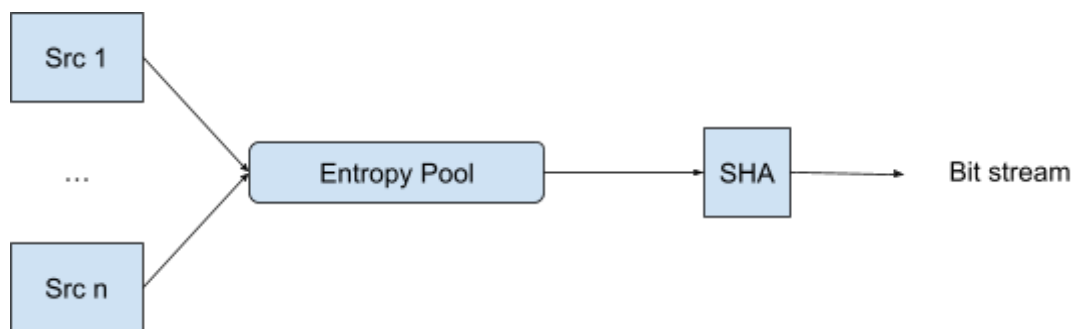
- Non soggetti a malfunzionamenti (*non hanno componenti fisici*).

Contro del hardware-based

- Vulnerabili all'osservazione.
- Vulnerabili alla manipolazione → Richiedono test periodici.

SWRNG - Linux

Vediamo ora l'approccio (*software-based*) usato nei sistemi operativi linux: Sia S_{rc_i} la i° fonte di randomicità.



PRBG - Pseudo-Random Bit Generators

$$G: \{0, 1\}^k \rightarrow \{0, 1\}^L, k \ll L$$

$G()$ è una funzione **veloce e deterministica** che prende in ingresso un seed s e restituisce uno stream di bit che sembrano casuali.



Il numero massimo di sequenze producibili dal PRBG è 2^k il quale è molto più piccolo delle sequenze casuali ottenibili con L bit (ossia 2^L): essendo il PRBG una funzione deterministica e avendo solo 2^k possibili input, avrò solo 2^k possibili output.

Ovviamente la sicurezza ne risente, poiché alcune sequenze a L bit non potranno mai comparire: il PRBG deve essere progettato in modo che non si possa capire il seed data la sequenza di uscita.

Sicurezza del PRBG

Ma la funzione $G()$ è sicura se rispetta questi 3 requisiti, che se sono rispettati allora le sequenze in uscita da PRG appaiano randomiche, anche se in realtà non lo sono.

Sicurezza computazionale del PRBG

k abbastanza grande da rendere infattibile un attacco esaustivo sul seed. Quindi segue lo stesso requisito di sicurezza computazionale della chiave di un cifrario.

Test del “prossimo bit” (forward unpredictability)

Dati i primi k bit di una sequenza casuale $b_1, b_2, \dots, b_k$ e supponendo che l'avversario non conosca il seed.

Non esiste alcun algoritmo eseguibile in tempo polinomiale che possa predire il bit b_{k+1} con una probabilità di successo maggiore del 50%.

Backward unpredictability

Supponendo che l'avversario conosca qualche sequenza prodotta in passato.

L'avversario da esse non può ricavare il seed.

Periodicità e grado di un PRGB

Ogni PRGB è un generatore pseudo-randomico, prima o poi i valori in uscita si ripeteranno; il quando accadrà dipende dal PRGB stesso, più tardi accade e migliore è la qualità del PRGB.

All'interno del periodo, la sequenza dei bit prodotta può considerarsi casuale.

Il grado di un PRG indica proprio quando accadrà questa cosa. Ad esempio LFSR è un PRG di grado m . quindi dopo $(2^m - 1)$ bit prodotti, la sequenza in uscita si ripeterà.

LCG - Linear Congruential Generator

$$s_0 = seed$$

$$s_{i+1} = |a * s_i + b|_m$$

Dove a, b, m sono costanti intere.

LCG ha delle buone proprietà statistiche

I bit in uscita sembrano veramente casuali.

LCG è prevedibile

Ha delle vulnerabilità dal punto di vista della sicurezza che rende questo generatore non adatto a scopi crittografici: dato un prefisso s_r, s_{r+1}, s_{r+2}

$$s_{r+1} = |a * s_r + b|_m$$

$$s_{r+2} = |a^* s_{r+1} + b|_m$$

Ho due incognite a, b e due equazioni, quindi risolvendo il sistema (*non lineare*) trovo a, b e quindi posso riprodurre tutti i bit successivi a s_{r+2} .

LFSR - Linear Feedback Shift Register

$$s_0 = s_0 \text{ (il seed)}$$

$$s_1 = |p_0 s_0|_2$$

$$s_2 = |p_1 s_1 + p_0 s_0|_2$$

...

$$s_m = |p_{m-1} s_{m-1} + \dots + p_1 s_1 + p_0 s_0|_2$$

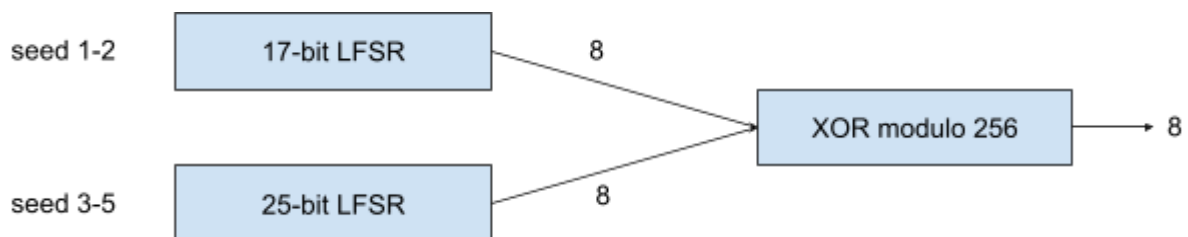
$$s_{m+1} = |p_{m-1} s_m + \dots + p_1 s_2 + p_0 s_1|_2$$

$$s_{i+m} = \left| \sum_{j=0}^{m-1} p_j^* s_{i+j} \right|_2$$

Content Scrambling System

Modo per migliorare le prestazioni di un LFSR.

Si usano in combinazione due LFSR.



CSPRNG - Cryptographically Secure PRNG

Un CSPRNG è un PRNG imprevedibile e quindi adatto ad applicazioni crittografiche. Per definizione un CSPRNG è tale se supera il “*test del prossimo bit*”.

Cifratura simmetrica e cifrari a blocchi

Per definizione un cifrario simmetrico è una coppia di funzioni del tipo:

$$E: \{0, 1\}^n \times \{0, 1\}^k \rightarrow \{0, 1\}^n$$

$$D: \{0, 1\}^n \times \{0, 1\}^k \rightarrow \{0, 1\}^n$$

Prendono in entrata un plaintext da n bit e una chiave da k bit, restituiscono un crittogramma da n bit.

Cifrario a blocchi perfetto (*true random cipher*)

- **Costruzione di un cifrario a blocchi perfetto?**
- **Costruzione cifrario perfetto, mostrare che non è possibile in pratica (permutazioni...); che permutazione implementa un cifrario?**

Dalla definizione di cifrario a blocchi si deduce che $|M| = |C|$ e quindi la cifratura è di fatto una permutazione.

Il set di permutazioni che lega ogni plaintext al relativo crittogramma è data dalla chiave.

Un cifrario a blocchi è perfetto solo se implementa **tutte** le possibili permutazioni: ossia $(2^n!)$ per avere ciò serve che $k \sim \log_2 2^n! \sim (n - 1,44) * 2^n$

Quindi la perfezione implica avere una chiave molto grande.

Dictionary attack

Tipologia di Known Plaintext Attack.

L'avversario conserva una tabella T composta da molte coppie $\langle p, c \rangle$, l'avversario ovviamente non conosce k e non conosce nemmeno come funziona il cifrario, ha solo molte corrispondenze $c \leftrightarrow p$.

Nel caso ideale, l'avversario ha tutte le possibili coppie, quindi 2^n coppie.

1. L'avversario intercetta un crittogramma c'
2. Se nella tabella T esiste una coppia con c' , allora il testo corrispondente è p' .

Questo attacco è molto potente ma è possibile solo se:

- La chiave non viene mai cambiata, ad ogni cambio la tabella diventa inutile perché le permutazioni sono cambiate.
- Se il numero di coppie possibili è piccolo, altrimenti diventa infattibile; ovvero se n è piccolo.

DES - Data Encryption Standard

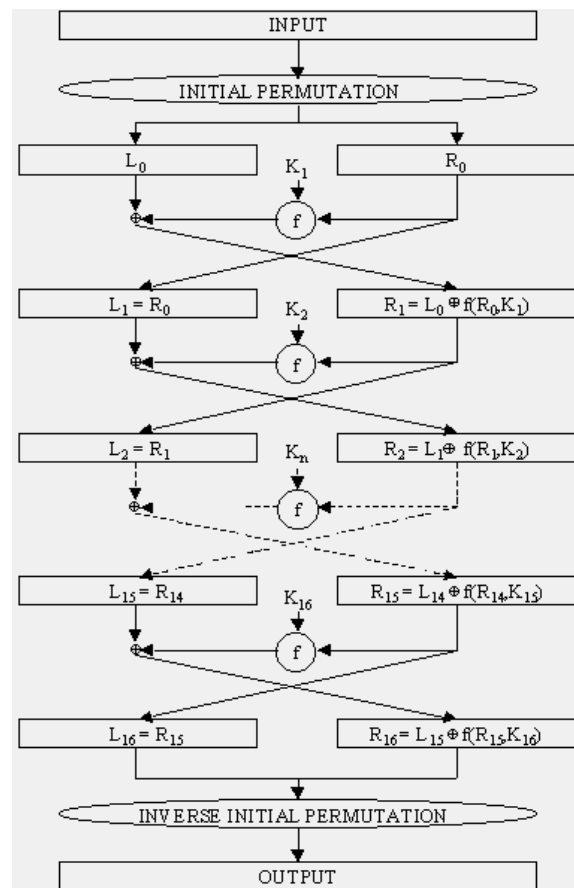
La chiave in entrata è di 64 bit, ma solo 56 sono effettivamente di sicurezza, gli altri 8 sono bit di parità.

Quindi la chiave effettiva è da considerarsi da 56 bit.

$$E: \{0, 1\}^{64} \times \{0, 1\}^{56} \rightarrow \{0, 1\}^{64}$$

$$D: \{0, 1\}^{64} \times \{0, 1\}^{56} \rightarrow \{0, 1\}^{64}$$

DES si basa su un sistema a 16 rounds.



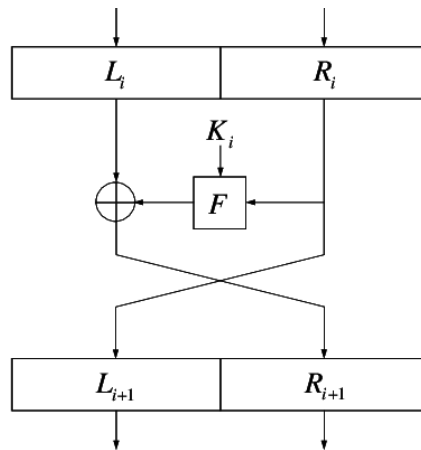
La prima cosa che si fa è espandere la chiave: da 56 bit di chiave si ottengono 16 sottochiavi da 48 bit (*una per round*).

Il plaintext passa da ogni round come in una catena di montaggio, passando dalle feistel networks.

Feistel Network

Ogni round è realizzato con una Feistel Network:

$$F_i: \{0, 1\}^n \rightarrow \{0, 1\}^n$$



Ognuna di queste funzioni ha lo scopo di realizzare **confusione** (*nascondere la relazione tra la chiave e il crittogramma*) e **diffusione** (*cambiare un simbolo del plaintext influenza tutti gli altri simboli del crittogramma*).

Di fatto possono essere considerate dei PRGB.

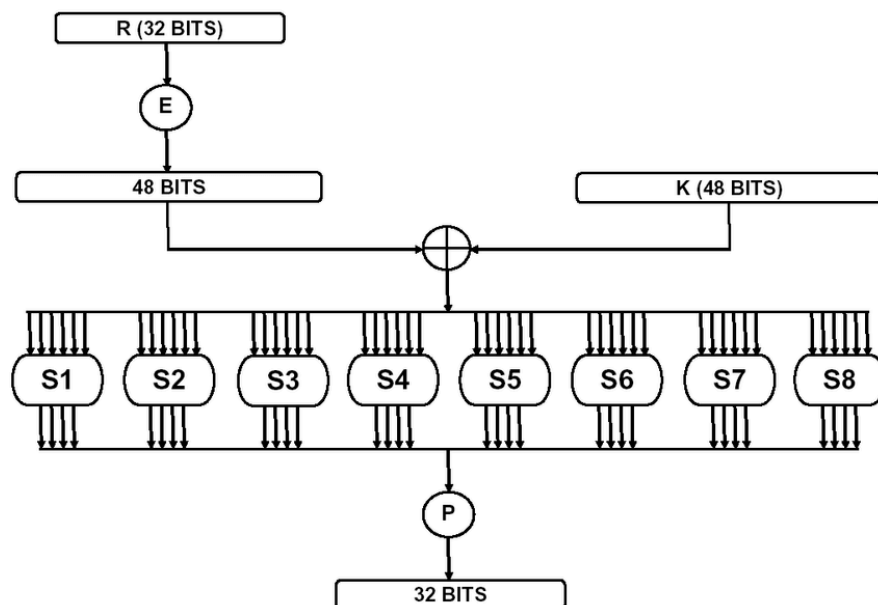
Ogni feistel network è invertibile; infatti il circuito di decifratura è lo stesso ma usato al contrario.

Ogni feistel network (*immagine sotto*) nel pratico è composta da:

1. Expansion box che porta R da 32 a 48 bit $\rightarrow$ Diffusione
2. Uno *xor* con la relativa round key K .
3. 8 S-Box , ogni S-box implementa una funzione non lineare (*le S-box sono sempre le stesse in ogni implementazione di DES, quindi possiamo vederle come delle semplici reti combinatorie*) $\rightarrow$ Confusione

$$S_i: \{0, 1\}^6 \rightarrow \{0, 1\}^4$$

4. Permutazione $P \rightarrow$ Diffusione



La combinazione di questi networks compongono una funzione invertibile.

Dove si usa DES ai giorni nostri?

DES ha molte implementazioni hardware che lo rendono velocissimo sia in cifratura che in decifratura, ma è anche insicuro.

Infatti è usato per cifrature leggere a bassa complessità e nei sistemi embedded (*grazie alle implementazioni hardware*).

Per questo viene usato sempre in meccanismi abbastanza ampi come HMAC o altro.

Per quale motivo 3DES è sicuro e 2DES no?

3DES e 2DES si ottengono concatenando DES.

2DES

$$c = E_{k_2}(E_{k_1}(m))$$

Diventa 2 volte più lento di DES e i bit della chiave passano da 56 a 112; ma l'attacco più semplice (*meet-in-the-middle*) ha complessità 2^{63} , avendo solo 63 bit di sicurezza è computazionalmente insicuro.

Meet-In-The-Middle

Dato $\langle c, p \rangle$ trovare k_1 e k_2 tali che $c = E_{k_2}(E_{k_1}(p))$.

Si esegue (*anche in parallelo*):

- Dato p , si calcola $z_i = E_{k_{1i}}(p)$

- Dato c , si calcola $z_j = D_{k_{2j}}(c)$

Appena trovo k_{1i} e k_{2j} tali che $z_i = z_j$ ho finito.

- Time Complexity $< 2^{63}$ (*fattibile ai giorni nostri*)
- Space Complexity $< 2^{56}$ (*molto spazio*)
 - Va memorizzata una entry per ogni chiave k_{1i} .

Quindi 2DES non ha vantaggi rispetto a DES, ma solo svantaggi (è *più lento*).

3DES

$$c = E_{k_1}(D_{k_2}(E_{k_3}(p)))$$

Diventa 3 volte più lento di DES e i bit della chiave passano da 56 a 168; ma l'attacco più semplice (*meet-in-the-middle*) ha complessità 2^{119} , avendo 118 bit di sicurezza è computazionalmente sicuro.

3DES invece non è così vulnerabile al meet-in-the-middle.

Dato $\langle c, p \rangle$ trovare k_1, k_2 e k_3 tali che $c = E_{k_2}(E_{k_1}(p))$

Si esegue (*anche in parallelo*):

- Dato p , si calcola $z_i = E_{k_{1i}}(p)$
- Dato c , si calcola $z_j = D_{k_{2j}}(D_{k_{3l}}(c))$

E quindi:

- Time Complexity $< 2^{118}$ (*infattibile ai giorni nostri*)
- Space Complexity $< 2^{56}$ (*molto spazio*)

Complessità di storage?

Importante solo in caso di precomputazione esaustiva. In quanto occorre memorizzare 2^{56} entry, una per ogni chiave k_{1i} .

La precomputazione esaustiva permette di ridurre la complessità di processazione al minimo, ma richiede molta memoria.

Complessità sui dati?

Importante solo in caso di:

- Crittoanalisi lineare
- Crittoanalisi differenziale

La precomputazione esaustiva permette di ridurre la complessità di processazione al minimo, ma richiede molta memoria.

Perchè non si va oltre a 3DES?

Il punto forte di DES è la velocità: il cifrario diventerebbe troppo lento; nonostante il grado di sicurezza aumenti ci sono migliori alternative con lo stesso grado di sicurezza.

AES - Advanced Encryption Standard

$$E: \{0, 1\}^{128} \times \{0, 1\}^{128/192/256/512} \rightarrow \{0, 1\}^{128}$$

$$D: \{0, 1\}^{128} \times \{0, 1\}^{128/192/256/512} \rightarrow \{0, 1\}^{128}$$

Anche AES si basa su round come DES ma ne ha di meno:

- 128 bits → 10 rounds.
- 192 bits → 12 rounds.
- 256 bits → 14 rounds.

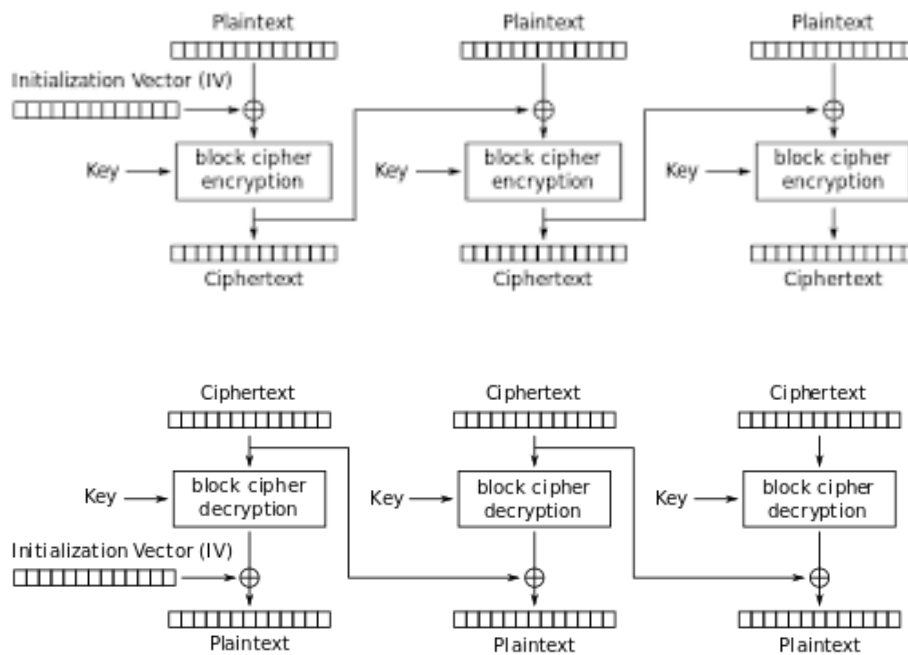
Encryption Mode

CBC - Cipher Block Chaining

$$C_i = E_K(P_i \oplus C_{i-1})$$

$$P_i = D_K(C_i) \oplus C_{i-1}$$

Cosa è lo schema CBC e spiegare la propagazione dell'errore

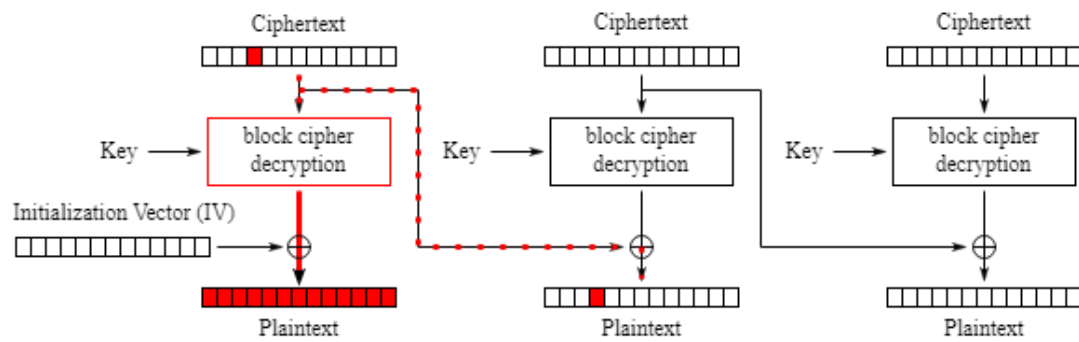


Riguardo la propagazione dell'errore in **cifratura**.

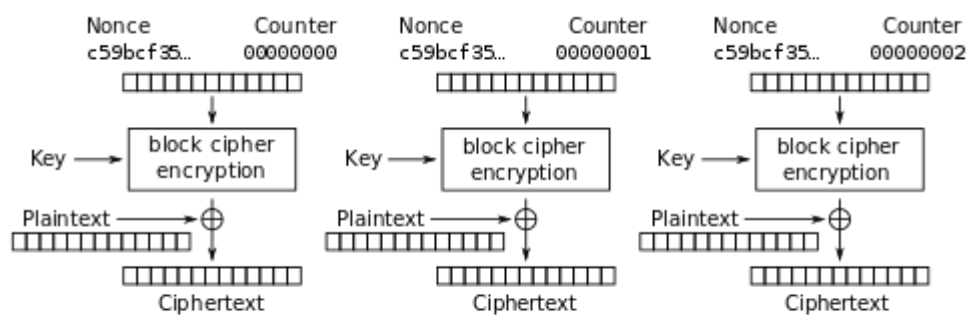
- Se c'è un errore nel IV, influenzerà tutti gli altri blocchi del crittogramma.
- Se c'è un errore in P_i , influenzerà tutti i blocchi successivi a C_i (*compreso*).

Riguardo la propagazione dell'errore in **decifratura**.

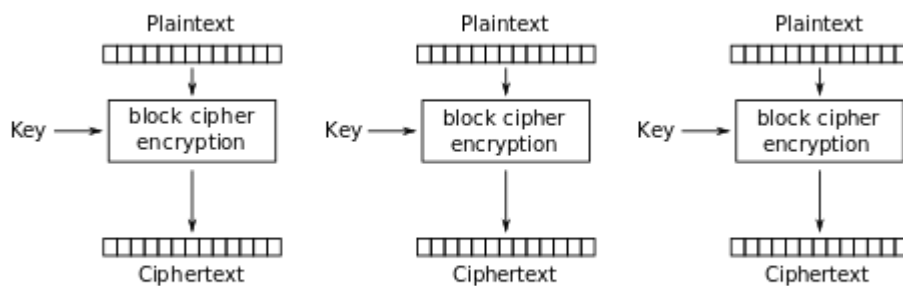
- Se c'è un errore nel IV, influenzerà solo il blocco P_1 , ossia il 1°.
- Se c'è un errore in C_i , influenzerà il blocco P_i e P_{i+1} .



CTR - Counter Mode Encryption



ECB - Electronic CodeBook



Crittografia asimmetrica

$$y = E_{P_k}(x)$$

$$x = D_{S_k}(y)$$

Con P_k la chiave pubblica e S_k la chiave privata.

Th - Un cifrario asimmetrico NON può essere perfetto

Il motivo è semplice: anche un avversario può produrre crittogrammi, poiché conosce la P_k .

1. L'avversario intercetta $y = E_{P_k}(x)$
2. L'avversario seleziona $x' : P_r[M = x'] \neq 0$ un messaggio possibile.
3. L'avversario calcola $y' = E_{P_k}(x')$ può farlo, la chiave P_k è pubblica.
4. Se $y = y' \Rightarrow x = x'$ e $P_r[M = x' | C = y] = 1$
5. Altrimenti $y \neq y' \Rightarrow x \neq x'$ e $P_r[M = x' | C = y] = 0$

Ad ogni tentativo l'avversario acquisisce conoscenza su x .

Small Plaintext Attack

Consideriamo x a 32 bit.

1. Alice calcola $y = E_{P_k}(x)$ e lo invia.
2. L'avversario intercetta y e calcola $\forall x_i \in M, y_i = E_{P_k}(x_i)$.
3. Prima o poi lo trova $\rightarrow$ sia $x^* : y = E_{P_k}(x^*)$ e $x^* \in M$.
4. L'avversario apporta una modifica $x^* \Rightarrow x'$
5. L'avversario calcola $y' = E_{P_k}(x')$ e lo invia al posto del crittogramma originale.

Questo attacco è possibile perché il punto (2) è computazionalmente fattibile: i bit del plaintext sono pochi (32) e quindi si può scorrere tutto lo spazio dei messaggi.

Questo problema si risolve con il **salting** $\rightarrow y = E_{P_k}(x || s)$

Supponendo che il salt sia a r bit e il plaintext sempre a 32 bit.

L'avversario oltre a tutti i plaintext deve anche provare tutti i possibili salt, quindi il punto (3) da complessità 2^{32} diventa 2^{32+r} .

RSA

Consideriamo i messaggi dei numeri interi minori di $n \rightarrow$
 $x, y \in \{0, 1, 2, \dots, n-2, n-1\}$

Generazione delle chiavi di RSA

Come funziona la generazione delle chiavi in RSA?

Complessità di ogni step dell'algoritmo di generazione delle chiavi di RSA?

1. Si scelgono p e q due numeri primi, diversi e molto grandi (~ 200 cifre decimali)
2. Si calcola $n = p * q$ detto "modulo".
3. Si calcola $\Phi(n) = (p - 1)(q - 1)$
4. Si sceglie randomicamente $e \in [2, \Phi(n) - 1]$ tale che $GCD(e, \Phi(n)) = 1$.

Ossia e deve essere coprimo con $\Phi(n)$; poiché (esiste un teorema) se $GCD(e, \Phi(n)) = 1$ allora d **esiste ed è unico**.

5. Si calcola $d \in [2, \Phi(n) - 1]$ tale che $|e * d|_{\Phi(n)} = 1$.

d è l'inverso di e in modulo $\Phi(n)$

6. p e q vengono eliminati.

Quindi le chiavi sono: $P_k = \langle e, n \rangle$ e $S_k = \langle d, n \rangle$

Dimostrazione della correttezza di RSA

Dati x e $y = |x^e|_n$ dimostriamo che: $x = |y^d|_n$

Noi sappiamo che per costruzione di $\langle e, d \rangle$ si verifica che:

$$|d * e|_{\Phi(n)} = |1|_{\Phi(n)} = 1 = |1 + t * \Phi(n)|_{\Phi(n)} \text{ con } t \text{ numero intero.}$$

Quindi:

$$|y^d|_n = |(x^e)_n^d|_n = |x^{ed}|_n = |x^{1+t\Phi(n)}|_n = |x * x^{t\Phi(n)}|_n = |x|_n * |x^{t\Phi(n)}|_n$$

Dobbiamo dimostrare che $|x^{t\Phi(n)}|_n = 1$

Se $GCD(x, n) = 1 \rightarrow$ **Th Eulero** $\rightarrow |x^{\Phi(n)}|_n = 1$

E quindi: $|x|_n * |x^{t\Phi(n)}|_n = |x|_n * (|x^{\Phi(n)}|_n)^t = |x|_n * |1|_n^t = x$

Se $GCD(x, n) \neq 1$ allora x è multiplo di p o di q (quale dei due in particolare è indifferente). Supponiamo che $x = r * p \rightarrow GCD(x, q) = 1 \rightarrow$ **Th Eulero** $\rightarrow |x^{\Phi(q)}|_q = 1$

$$|x^{\Phi(n)}|_q = |x^{(p-1)(q-1)}|_q = |x^{(p-1)\Phi(q)}|_q = (|x^{\Phi(q)}|_q)^{(p-1)} = |1^{p-1}|_q = 1$$

Quindi in questo caso: $|x^{\Phi(n)}|_n = |1 + t * q|_n$

$$|x|_n * |x^{t\Phi(n)}|_n = |x + x * t * q|_n = |x + (r * p) * t * q|_n = |x + r * t * n|_n = |x|_n = x$$

Performance RSA

Calcolo di e

Si parte con $e = 0$

1. $e++$
2. Si usa l'equazione di Diophantine: $GCD(e, \Phi(n)) = s * \Phi(n) + t * e$
3. Se $GCD(e, \Phi(n)) = 1 \rightarrow$ **fine**
4. Altrimenti si torna al punto (2)

Questo algoritmo ha complessità $\log(n)$

Calcolo di p e q

Per entrambi il procedimento è lo stesso:

1. $p = rand()$
2. p è pari $\rightarrow$ **torna al punto (1)**
3. p non è primo $\rightarrow$ **torna al punto (1)**
4. **Fine**

Il punto (3) è un test di primalità (*test di Miller-Rabin o il test di Fermat*) che richiede diverse operazioni, quindi per ottimizzare si mette il punto (2): si testano solo i numeri dispari.

Sia $P_i(x)$ il numero di numeri primi minori di x

Se $x \gg 1 \Rightarrow P_i(x) \sim \frac{x}{\ln(x)}$ e sono distribuiti in maniera uniforme in $[2, x - 1]$

La probabilità di trovare un numero primo in $[0, x] \Rightarrow \sim \frac{2}{\ln(x)}$; quindi mi aspetto
 $\sim \frac{\ln(x)}{2}$ tentativi

Operazioni in modulo

Supponiamo che n , a e b siano tutti su k bits.

$$|a + b|_n \sim O(k)$$

$$|a - b|_n \sim O(k)$$

$$|a * b|_n \sim O(k^2)$$

$$|\frac{a}{b}|_n \sim O(k^2)$$

$$|a^{-1}|_n \sim O(k) \rightarrow \text{Complessità del calcolo di } d.$$

$$|a^b|_n \sim O(k^3) \rightarrow \text{Bisogna ottimizzarlo.}$$

Square-and-multiply

Sia $[x_{k-1}, x_{k-2}, \dots, x_2, x_1, x_0]$ la rappresentazione in binario di x

$$\begin{aligned} |a^x|_n &= |a^{x_{k-1}2^{k-1}} * a^{x_{k-2}2^{k-2}} * \dots * a^{x_12} * a^{x_0}|_n = |(a^{x_{k-1}2^{k-2}} * a^{x_{k-2}2^{k-3}} * \dots * a^{x_1})^2 * a^{x_0}|_n \\ &= |((a^{x_{k-1}2^{k-3}} * a^{x_{k-2}2^{k-4}} * \dots)^2 * a^{x_1})^2 * a^{x_0}|_n = \dots = |(((a^{x_{k-1}})^2 * a^{x_{k-2}})^2 * \dots)^2 * a^{x_0}|_n \end{aligned}$$

Se $x_i = 0$ allora $a^{x_i} = 1$ e quindi tolgo una esponenziazione alla 2° .

Quante operazioni faccio?

$$\#MUL + \#SQ = \text{"Numero di bit a 1 in } x" + k$$

Supponendo un caso medio, in cui solo il 50% dei bit di x è a 1

$$\#MUL + \#SQ = \frac{k}{2} + k = 1536$$

Quindi con $k = 1024$ faccio 1536 moltiplicazioni su 1024 bit.

Altre ottimizzazioni

Possiamo usare un e piccolo, tanto è pubblico e ciò semplifica la cifratura, ma comunque mantenendo $x^e < n$, altrimenti $y = |x^e|_n = x^e$ e quindi $x = \sqrt[e]{y}$.

Invece un d piccolo è sconsigliato $\rightarrow$ Sempre avere $size(d) = size(n)$

Fast Decrypt

1. Calcolo $x_p = |x|_p$ e $x_q = |x|_q$
2. Calcolo $y_p = |x_p^{d_p}|_p$ con $d_p = |d|_{p-1}$ (*precalcolato*)
3. Calcolo $y_q = |x_q^{d_q}|_q$ con $d_q = |d|_{q-1}$ (*precalcolato*)
4. $y = |[q * C_p] * y_p + [p * C_q] * y_q|_n$

con $C_p = |q^{-1}|_p$ (*precalcolato*)

con $C_q = |p^{-1}|_q$ (*precalcolato*)

Ho un peggioramento della sicurezza: p e q non possono essere eliminati dopo la generazione delle chiavi. Se anche solo uno dei due venga svelato all'avversario, potrebbe fattorizzare n e trovare facilmente d .

Plain-RSA

Perché non si usa in pratica? (è insicuro contro adaptive-chosen ciphertext attack).

Soluzione ai problemi? (padding per RSA e salting)

Plain-RSA è malleabile

come viene sfruttata questa caratteristica: (Adaptive-Chosen Ciphertext Attack) ?

1. Alice trasmette $y = |x^e|_n$
2. L'avversario riceve y e sceglie $s : GCD(s, n) = 1$
3. L'avversario calcola $y' = |s^e * y|_n$ e lo invia a Bob.
4. Bob riceve $y' \Rightarrow x' = |y'^d|_n = |s^{ed} * y^d|_n = |s * x|_n$

Bob non se ne accorge. Questo problema si risolve con il **OAEP (Padding per RSA)**.

Adaptive-Chosen Ciphertext Attack

L'avversario vuole conoscere il plaintext corrispondente a y , ma Bob decifra e rinvia tutto quello che riceve, eccetto y .

1. L'avversario sceglie $s : GCD(s, n) = 1$ e calcola $y' = |s^e * y|_n$
2. Bob riceve $y' \Rightarrow x' = |y'^d|_n = |s^{ed} * y^d|_n = |s * x|_n$ e lo invia all'avversario.

Bob lo invia perché ha ricevuto $y' \neq y$.

3. L'avversario calcola $x = \frac{x'}{s}$

Plain-RSA è deterministico

Dato un x viene restituito sempre lo stesso y ; ciò lo rende vulnerabile ad attacchi di tipo dictionary.

Questo problema si risolve con il **salting**.

Small Plaintext Attack

Se $x^e < n$ allora $y = x^e$ e quindi $x = \sqrt[e]{y}$

Oppure ho comunque problemi se lo spazio dei messaggi è piccolo poiché si può fare un attacco a forza bruta sui messaggi invece che sulla chiave.

Questo problema si risolve con il **OAEP (Padding per RSA)**.

Common Modulus Attack

Tutte le chiavi private degli utenti usano lo stesso modulo. L'avversario si registra e riceve una regolare chiave dal server.

Da una chiave privata si può facilmente fattorizzare n e quindi trovare anche le chiavi private degli altri client.

Attacco del teorema cinese del resto

Supponiamo che e sia molto piccolo (tale che $x^e < n$) e che a tutti i client venga inviato $y_i = |x^e|_{n_i}$ e n_i tutti coprimi tra loro.

Usando il teorema cinese del resto: faccio un sistema lineare.

$$z = |y_1|_{n_1}$$

$$z = |y_2|_{n_2}$$

$$z = |y_3|_{n_3}$$

$$\text{Se } x < n_i \Rightarrow x^3 < n_1 * n_2 * n_3 \Rightarrow z = |x^3|_{n_1 * n_2 * n_3} \Rightarrow x = \sqrt[3]{z}$$

Questo problema si risolve con l'aggiunta del salt $\rightarrow y_i = |x_i^e|_{n_i}$ e con esponenti abbastanza grandi.

RSAP - RSA Problem (DLP)

Dato y e la $P_k = \langle e, n \rangle$, trovare x tale che $y = |x^e|_n$

Se p e q sono noti $\rightarrow$ RSAP è semplice da risolvere poiché la fattorizzazione di n diventa facile.

$$DLP \leq_p \text{Fattorizzazione}(n)$$

Se la fattorizzazione di n è facile, lo è anche RSAP. La fattorizzazione è un problema che ad oggi è sub-esponenziale (*ossia l'algoritmo più efficiente che lo risolve ha complessità $2^{\frac{t}{2}}$ con t il numero di bit di n*).

Th1

Calcolare d partendo dalla P_k è equivalente a fattorizzare n .

- Se p e q sono noti $\rightarrow$ La fattorizzazione di n è facile $\rightarrow$ trovare d è facile.
- Se d è nota $\rightarrow$ La fattorizzazione di n è facile (*principio usato nel common modulus*).

Th2

Se n è composto (*non primo*), allora calcolare $x = |\sqrt[e]{y}|_n$ è equivalente a fattorizzare n

Th3

Calcolare $\Phi(n)$ partendo da n senza sapere p e q è equivalente a fattorizzare n

RNG: proprietà per la sicurezza?

Deve essere più difficile della fattorizzazione di n . Per avere ciò:

- $size(n) = size(d)$
- $size(q) \approx size(p)$
- $size(q) \ll size(d)$
- $size(p) \ll size(d)$

Diffie-Hellman Key Exchange (DHKE)

Diffie-Hellman, Operazioni su Z_p^ e MiM?*

Diffie-hellman (per scambio di chiavi) ?

Perché è ritenuto rivoluzionario?

Protocollo rivoluzionario perché non prevedeva che A e B condividessero un segreto a priori, quindi non segreti a lungo termine.

Sia $Z_p^* = [0, \dots, p-1]$ tali che $GCD(i, p) = 1$

Sia $g \in Z_p^*$ detto “generatore” di un sottogruppo ciclico di Z_p^* .

Sia p un numero primo. Per generare p si usa lo stesso criterio che si usa per n in RSA.

1. Alice calcola $a = rand()$
2. Bob calcola $b = rand()$
3. Alice invia a Bob

M1 $A \rightarrow B$: “Alice”, $y_A = |g^a|_p$

4. Bob invia ad Alice

M2 $B \rightarrow A$: “Bob”, $y_B = |g^b|_p$

5. Alice calcola $K_{AB} = |y_B^a|_n = |g^{ab}|_n$

6. Bob calcola $K_{AB} = |y_A^b|_n = |g^{ab}|_n$

Proprietà di Z_p^*

Perché si lavora in un sottogruppo ciclico Z_p^* ?

Lavorare in un sottogruppo ciclico garantisce che le proprietà matematiche necessarie per la sicurezza e l'efficienza del protocollo siano soddisfatte.

Questo rende il protocollo resistente a diversi tipi di attacchi e più pratico da implementare.

- Consentono delle interessanti proprietà matematiche che aumentano l'efficienza computazionale.

$$|\log_g(\beta\gamma)|_p = |\log_g(\beta) + \log_g(\gamma)|_p$$

$$|\log_g(\beta^s)|_p = s * |\log_g(\beta)|_p$$

- Usare un sottogruppo ciclico riduce il rischio di alcuni tipi di attacchi, come quelli basati sulla fattorizzazione del modulo p o su proprietà aritmetiche specifiche del gruppo completo.
- In un sottogruppo ciclico, gli esponenti sono distribuiti uniformemente, il che significa che ogni possibile valore di g^k è ugualmente probabile. Questo contribuisce alla sicurezza del protocollo contro analisi statistica e attacchi basati su pattern ripetitivi.

Esponenziazione discreta

Dati $x, y \in Z_p^*$ il calcolo di $y = |g^x|_p$ è facile (*le esponenziazioni in modulo si fanno con square-and-multiply*).

Logaritmo discreto

Dati $y \in Z_p^*$ il calcolo di $x \in Z_p^*$ tale che $y = |g^x|_p$ è difficile.

Il protocollo DH si basa sulla difficoltà computazionale del problema del logaritmo discreto. In un sottogruppo ciclico, questo problema è ben definito e difficile da risolvere, rendendo il protocollo sicuro.

La sicurezza del protocollo dipende dall'impossibilità pratica di trovare k dati g e g^k

Ciclicità

Un sottogruppo Z_p^* è ciclico se contiene un elemento α con ordine

$$\text{ord}(\alpha) = k \leq |Z_p^*| = p - 1 \text{ tale che } |\alpha^k|_p = 1.$$

Z_p^* è **ciclico**; dato $g = 2$ e $p = 11$, calcoliamo $|g^i|_p$ all'incremento di i .

$$|2^0|_{11} = 1$$

$$|2^1|_{11} = 2$$

$$|2^2|_{11} = 4$$

...

$$|2^{10}|_{11} = 1$$

$$|2^{11}|_{11} = 2$$

Si ripete. Invece riguardo alle operazioni ci sono delle proprietà: Le potenze di un elemento primitivo g definiscono una permutazione degli elementi di Z_p^* .

Teorema 8.2.3 - Fermat Little Theorem

Sia G un gruppo finito $\rightarrow \forall a \in G, a^{|G|} = 1$ e $|G|$ è multiplo di $\text{ord}(a)$.

Teorema 8.2.4

Sia G un gruppo finito, allora:

- Il numero di elementi primitivi in G è pari a $\Phi(|G|)$.
- Se $|G|$ è un numero primo $\rightarrow$ Tutti gli elementi $a \in G$ e $a \neq 1$ sono primitivi.

Teorema 8.2.5 - Cyclic Subgroup Theorem

Sia G un gruppo ciclico.

$\forall a \in G$ con $\text{ord}(a) = s$ è l'elemento primitivo di un sottogruppo ciclico di s elementi.

Esempio

Sia $Z_{11}^* \rightarrow 11$ è un numero primo, quindi per il **Th 8.2.4** ogni elemento $a \neq 1$, $a \in Z_{11}^*$ è un elemento primitivo.

Supponendo $a = 3 \rightarrow s = \text{ord}(3) = 5$

Quindi il sottogruppo ciclico relativo ad a è $H = \{1, 3, 4, 5, 9\}$

Teorema 8.2.6 - Teorema di Lagrange

Sia H un sottogruppo di $G \rightarrow |G|$ è multiplo di $|H|$.

Teorema 8.2.7 - Cyclic Subgroup Theorem

Sia G un gruppo ciclico e finito di ordine n .

Sia a un generatore di G .

$\forall k$ multiplo di n esiste un sottogruppo ciclico H di G di ordine k .

Per ognuno di questi sottogruppi:

- Il generatore di H è generato da $a^{\frac{n}{k}}$.

- H consiste di elementi $a \in G$ tali che $|a^k|_p = 1$

Esempio

Sia $Z_{11}^* \rightarrow 11$.

Sia $a = 8$, $k = 2 \rightarrow \beta = |8^{\frac{10}{2}}|_{11} = |10|_{11}$ è generatore di H di ordine $k = 2$.

Sicurezza Diffie-Hellman

La sicurezza di Diffie-Hellman è analoga a quella di RSA: Dato y_A e y_B trovare K_{AB} senza sapere a e b .

$$DHP \leq_p DLP \leq_p \text{Fattorizzazione}(n)$$

Vantaggi tratti dai gruppi ciclici

In gruppi ciclici non esistono algoritmi efficienti conosciuti per risolvere il logaritmo discreto, il che rende il protocollo sicuro contro attacchi brute force o algoritmi noti.

In un sottogruppo ciclico, la difficoltà della risoluzione del logaritmo discreto è maggiore per i seguenti motivi:

- **Struttura matematica:** La struttura ben definita e uniforme del sottogruppo ciclico rende il problema del logaritmo discreto difficile da sfruttare per gli attaccanti.
- **Ordine grande del sottogruppo:** Se il sottogruppo ha un ordine sufficientemente grande, gli attacchi brute force diventano impraticabili.
 - Un ordine grande implica un vasto numero di possibili esponenti, rendendo gli attacchi brute force inefficaci.
 - La difficoltà nel prevedere i risultati delle operazioni nel gruppo aumenta con la grandezza del sottogruppo, complicando ulteriormente la risoluzione del logaritmo discreto.
 - Algoritmi come il baby-step giant-step o la rho di Pollard richiedono un tempo di calcolo che cresce significativamente con l'aumentare dell'ordine del sottogruppo, rendendoli inefficaci per gruppi molto grandi.
- **Mancanza di fattori semplici:** Un sottogruppo scelto opportunamente evita ordini con piccoli fattori primi, riducendo l'efficacia di algoritmi di attacco specifici.

- **Assenza di punti deboli:** Nei sottogruppi ciclici, non ci sono elementi con ordini inferiori che potrebbero facilitare attacchi.

MiM in DHKE

1. Operazioni preliminari:
 - Alice calcola $a = rand()$
 - Bob calcola $b = rand()$
 - L'avversario calcola $c = rand()$

2. Alice invia a Bob

"Alice", $y_A = |g^a|_p$

3. L'avversario riceve intercetta il messaggio.

4. L'avversario invia a Bob

"Alice", $y_C = |g^c|_p$

5. Bob invia ad Alice

"Bob", $y_B = |g^b|_p$

6. L'avversario riceve intercetta il messaggio.

7. L'avversario invia ad Alice

"Bob", $y_C = |g^c|_p$

8. Alice calcola $K_{AC} = |y_C^a|_n = |g^{ac}|_n$

9. Alice calcola $K_{CB} = |y_C^b|_n = |g^{cb}|_n$

Questo attacco si evita utilizzando l'autenticazione e quindi i certificati.

Firma Digitale

Un sistema di firma digitale è composto da 3 algoritmi:

- $G() \rightarrow$ Usato per generare due chiavi di un cifrario asimmetrico $\langle P_k, S_k \rangle$
- $\sigma = S(S_k, x) = E_{S_k}(x) \rightarrow$ Usato per generare la firma su un oggetto x .

- $V(P_k, \sigma, x) = (D_{P_k}(\sigma) == x) \rightarrow$ Restituisce “vero” o “falso”, quindi è usato per verificare la firma σ su un oggetto x .

Una firma digitale deve garantire:

- **Consistenza:** $\forall x \forall \langle P_k, S_k \rangle, V(P_k, x, S(S_k, x)) = 1$
- **Sicurezza:** Anche dopo aver letto n firme diverse su n messaggi, prodotte con la stessa S_k l'avversario non può essere in grado di creare una firma valida ma fasulla su un nuovo messaggio.
- **Non ripudiabilità:** Convincere un esterno o se stessi in un istante t_1 , dell'integrità e autenticità di un messaggio inviato a tempo $t_0 < t_1$.
- **Autenticazione:** Convincere un esterno o se stessi dell'integrità e autenticità di un messaggio in un tempo t_0 .

Secret Splitting

Divido il messaggio in 2 pezzi, senza entrambi non si può ricostruire il messaggio di partenza. Un esempio:

$$M = R \oplus L$$

Con:

- $R = rand()$
- $L = M \oplus R$

Bit commitment

Siano p un numero primo e $g \in Z_p^*$ un generatore.

1. Alice genera un numero $b \in [0, p - 1]$
2. Alice calcola il commitment $c = |g^b|_p$ e lo pubblica.
3. Bob per verificare che Alice non abbia “barato”, basta che calcola c e lo confronta con quello pubblicato originariamente da Alice nel punto (2).

Double spending

Un cliente non può spendere il soldo digitale molteplici volte.

Un mercante non può depositare il soldo digitale molteplici volte.

Il sistema deve evitare questa cosa e identificare il fraudolento.

$$Coin = \langle coin, IdentityString, h(coin, identitystring)^d \rangle$$

- *coin* è una quantità randomica.

La identity string la posso vedere come una sequenza di 100 bit:

$$IdentityString = [I_0, I_1, \dots, I_{100}]$$

Ad ogni bit della identity string applico il Secret Splitting: $I_i \rightarrow \langle I_{iL}, I_{iR} \rangle$

Ad ognuno applico il commitment: $\langle I_{iL}, I_{iR} \rangle \rightarrow \langle C_{iL}, C_{iR} \rangle$

Ogni coppia $\langle C_{iL}, C_{iR} \rangle$ è differente dall'altra.

1. Alice invia $H(\text{coin}_i, \text{identityString}_i) * b_i^e, 1 \leq i \leq 100$ alla banca.
2. La banca sceglie 99 di questi messaggi a random e chiede ad Alice di rivelarli.
3. Alice invia $H(\text{coin}_i, \text{identityString}_i), 1 \leq i \leq 99$ alla banca.
4. La banca chiede di mostrare le 99 identity string relative ai 99 messaggi rivelati.
5. Alice invia $\langle I_{jL}, I_{jR} \rangle, 1 \leq i \leq 99$ alla banca.

Alla fine del punto (5) la banca è convinta al 99% che il commitment non aperto (*il k°*) contiene l'identità di Alice.

6. La banca invia $H(\text{coin}_k, \text{identityString}_k)^d * b_k$ ad Alice. Alice userà questo coin per attestare l'autenticità del soldo digitale.

$$H(\text{coin}_k, \text{identityString}_k)^d * b_k \rightarrow * b_k^{-1} \rightarrow H(\text{coin}_k, \text{identityString}_k)^d$$

$H(\text{coin}_i, \text{identityString}_i)^d$ è il coin_i autenticato.

Quando Alice spende:

1. Alice invia a Bob:

$$\langle \text{coin}, \text{IdentityString}, h(\text{coin}, \text{identity string})^d \rangle$$

2. Bob chiede ad Alice le identity strings.

3. Alice invia a Bob:

$$\langle I_{iL}, I_{iR} \rangle = I_{ib_i}, 1 \leq i \leq 100.$$

4. Bob invia alla Banca:

$\langle coin, IdentityString, H(coin, identitystring)^d \rangle, I_{ib_i}, 1 \leq i \leq 100$

5. La banca verifica la firma digitale.

6. Se il *coin* non è ancora stato speso $\rightarrow$ OK

7. Altrimenti è un double spending!

- Se le I_{ib_i} sono le stesse $\rightarrow$ Bob è fraudolento.
- Altrimenti $\rightarrow$ Alice è fraudolenta, lo verifica grazie all'identity string che la banca ha tenuto segreta.

Certificati digitali

Quali sono le informazioni minime che ci aspettiamo di trovare dentro un certificato?

$Cert_A = \langle "Alice", P_{KA}, L_A, S_{CA}("Alice" | P_{KA} | L_A) \rangle$

- Identificatore dell'utente $\rightarrow$ "Alice"
- Chiave pubblica dell'identificato $\rightarrow P_{KA}$
- Periodo di validità del certificato $\rightarrow L_A$
- Firma digitale della CA $\rightarrow S_{CA}("Alice" | P_{KA} | L_A)$

Un certificato prova che una certa chiave pubblica appartiene ad un certo soggetto. I certificati sono rilasciati e distribuiti dalle Certification Authority (CA).

Questa struttura previene gli attacchi MIM, nel quale l'avversario dà la sua chiave pubblica invece che quella del server.

Rilascio di un certificato (CA-generated keys)

1. Alice invia la richiesta alla CA.

M1 $A \rightarrow CA: req, "Alice"$

2. Viene creato un canale sicuro tra Alice e la CA (tramite un key-exchange protocol).

Quindi tra la CA e Alice c'è una chiave simmetrica condivisa K .

M2 $A \rightarrow CA: \{req, documenti\}_K$

3. La CA verifica l'identità di Alice (*documenti, etc*).

4. La CA genera $\langle P_{KA}, S_{KA} \rangle$, genera L_A , calcola $S_{CA}(\text{"Alice"} | P_{KA} | L_A)$ e infine compone $Cert_A$.

5. La CA invia ad Alice il suo certificato:

M3 CA $\rightarrow$ A: $\{Cert_A, S_{KA}\}_K$

Backup della chiave privata

Se non si recupera la S_k alcuni dati diventerebbero inaccessibili. Ma il recupero di una chiave privata permetterebbe di leggere dati anche dopo la scadenza di questa chiave.

Ma cancellare la S_k dopo la scadenza comporta la perdita della non-ripudiabilità poiché quella chiave potrebbe essere riutilizzata altrove.

Threshold Crypto

Il segreto, (S_k) viene diviso in n frammenti e divisi in n posti diversi. Almeno t frammenti sono necessari per ricostruire il segreto.

Esempio

($t = 2, n$) Sistema in cui il segreto è un punto P in un piano cartesiano (*quello che si interseca nell'asse y*).

Si crea una retta passante per P e si identificano n punti di questa retta. I punti sono i frammenti, e per recuperare l'equazione della retta bastano $t = 2$ punti.

Verifica di un certificato

Quali sono i controlli da fare prima di utilizzarlo?

When is a certificate considered valid?

Da fare una sola volta (*prelevare il certificato della CA*):

1. Bob ottiene la P_{KCA}
2. Bob verifica la validità della P_{KCA}

Da fare ad ogni sessione:

3. Bob richiede il certificato di Alice alla CA.
4. Bob **verifica** che il certificato sia integro verificando la firma usando P_{KCA} .
5. Bob **verifica** che il certificato sia valido verificando L_A (*verifica che non sia scaduto*).
6. Bob **verifica** che il certificato non sia stato revocato.
 - CRL
 - OCSP

7. Se tutto va bene, Bob accetta (*poiché si fida della CA*) che P_{KA} è la chiave pubblica di Alice.

CRL - Certificate Revocation List

Lista periodicamente aggiornata dalla CA che contiene tutti i certificati che sono stati revocati prima della loro scadenza.

Passata la data di scadenza, il certificato revocato viene rimosso (*per tenere la lista più piccola possibile*), inoltre un certificato scaduto fallisce sempre la verifica.

La CRL è inviata dalla CA, ovviamente il messaggio che la trasporta è firmato assieme ad un timestamp che ne prova la freschezza.

Per ogni certificato contiene:

- Numero seriale.
- Data di revocazione.
- Motivo.

OCSP - Online Certificate Status Protocol

Scambio di messaggi CA-Utente che si effettuano durante il controllo di validità di un certificato?

M1 $A \rightarrow CA: \quad t_A, \text{"Numero Seriale Certificato di Bob"}$

M2 $CA \rightarrow A: \quad \{t_A, response\}_{S_{KCA}}$

Il timestamp inviato da A (t_A) svincola la CA dal requisito della sincronizzazione del clock. Poiché la validità verrà calcolata basandosi sul timestamp inviato da Alice.

Trust Model

Come funziona la cross certification?

Centralizzato

Ho una sola CA (CA_R), che rilascia tutti i certificati.

Gli utenti si fidano della CA_R .

Non centralizzato

Non si può usare una sola CA. Spesso esistono delle CA subordinate (CA_A e CA_B) legate alla CA_R

Gli utenti si fidano della CA_R .

CA_A e CA_B rilasciano i certificati rispettivamente all'utente A e all'utente B .

- Il certificato di B che verrà inviato ad A è del tipo: $Cert_{CA_R}\{CA_B\}, Cert_{CA_B}\{B\}$
- Il certificato di A che verrà inviato ad B è del tipo: $Cert_{CA_R}\{CA_A\}, Cert_{CA_A}\{A\}$

Ossia anche la CA che ha certificato B ha un certificato rilasciato dalla CA_R .

Cross Verification

CA_A e CA_B rilasciano i certificati rispettivamente all'utente A e all'utente B .

L'utente A si fida di CA_A .

L'utente B si fida di CA_B .

- Il certificato di B che verrà inviato ad A è del tipo: $Cert_{CA_A}\{CA_B\}, Cert_{CA_B}\{B\}$
- Il certificato di A che verrà inviato ad B è del tipo: $Cert_{CA_B}\{CA_A\}, Cert_{CA_A}\{A\}$

Browser Verification

L'utente ha a disposizione una lunga lista di CA e a priori si fida di tutte.

Funzioni di hash

Le proprietà di una funzione hash crittograficamente sicura?

Come funziona il guessing attack (funzionamento e complessità)?

Come funziona il birthday attack (funzionamento e complessità)?

Schema di Merkle-damgaard?

Come si realizza la funzione $h()$ in Merkle-damgaard?

Si hanno vantaggi passando ad hash più grandi da hash piccoli?

Cosa sono le funzioni di compressione?

Le proprietà di una funzione Hash

$$H: \{0, 1\}^* \rightarrow \{0, 1\}^n$$

Compressione

Il numero di bit in entrata è indefinito (*), ma il numero di bit n in uscita è fisso; di solito $* \gg n$.

Pre-image resistance

Dato un output di $H()$, $y \in \{0, 1\}^n$ deve essere computazionalmente infattibile trovare $x: y = H(x)$.

Ossia trovare un valore che restituisce quello stesso valore come output della funzione di hash.

2° Pre-image resistance

Dato un output di $H()$, $y = H(x)$ deve essere computazionalmente infattibile trovare $x' \neq x: y = H(x')$.

Ossia, dato x trovare una collisione.

Strong collision resistance

Dato x deve essere computazionalmente infattibile trovare $x' \neq x: H(x) = H(x')$

Questa proprietà implica anche la 2° preimage resistance ma non implica la 1° preimage resistance

Blackbox attacks

Tipologia di attacco in cui si tenta di trovare una collisione senza sfruttare le proprietà intrinseche della funzione di hash.

Guessing attack

Trovare una 2° preimage: dato $y = H(x)$ trovare $x': H(x) = H(x')$

Questo attacco richiede in media 2^n valutazioni della funzione $H()$, dando per scontato che $rand()$ e $H()$ siano molto efficienti.

repeat

$$x_1 \leftarrow rand()$$

until $H(x) == H(x_1)$

Birthday attack

Cercare due messaggi con lo stesso valore di hash.

1. L'avversario sceglie $N = 2^{\frac{n}{2}}$ messaggi $x_1, x_2, \dots, x_N$.
2. L'avversario calcola $t_i = H(x_i) \forall i \in [1, N]$
3. L'avversario cerca i, j tale che $t_j = t_i$ e $i \neq j$

La probabilità che al punto 3 si trovi almeno una collisione è del 50%.

Esempio più pratico: Dato $y = H(x)$ alterare in modo controllato x' in modo che $H(x) = H(x')$.

- $x_1 \rightarrow \text{"Manda 100\$ ad Alice"}$
- $x_2 \rightarrow \text{"Manda 1000\$ ad Alice"}$

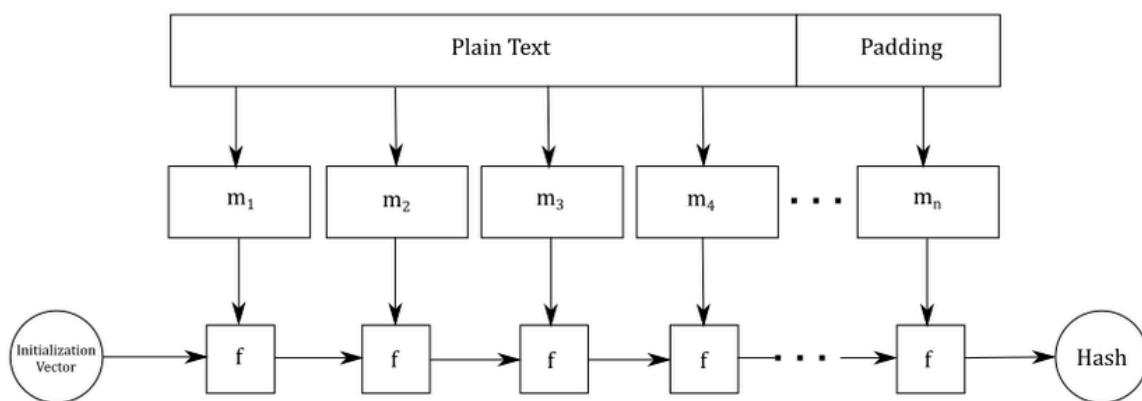
Altero x_2 aggiungendo spazi bianchi in fondo fino a che $H(x_1) = H(x_2)$

repeat

$$x_2 = \langle x_2 \mid " " \rangle$$

until $H(x_2) == H(x_1)$

Schema Merkle-Damgaard



Il messaggio viene diviso in blocchi di dimensione fissa.

- Se l'ultimo blocco è più piccolo si aggiunge padding fino al raggiungimento della dimensione del blocco.
- Se l'ultimo blocco non ha bisogno di padding (*dimensione messaggio è multipla della dimensione del blocco*) allora si aggiunge un blocco di solo padding.

Le funzioni $h()$ (nell'immagine è $f()$) sono dette **funzioni di compressione**.

Ogni valore in uscita dalle $h()$ sono dette "*chaining variables*".

Come realizzare le $h()$

Se le $h()$ sono resistenti alle collisioni, allora anche $H()$ lo è. Quindi lavoriamo sulle $h()$ per quanto riguarda la sicurezza.

Hash con cifrari a blocchi

Per realizzare le $h()$ possiamo usare dei cifrari a blocchi come DES o AES.

$\dim(\text{blocco}) = \dim(\text{digest})$ e quindi la resistenza alle collisioni può non pervenire nel caso in cui i blocchi siano piccoli (*poiché brutare i blocchi è fattibile*); quindi conviene usare cifrari con blocchi grandi.

Schema di Davies-Meyer

$$H_{i+1} = h(H_i) = H_i \oplus E_{m_i}(H_i)$$

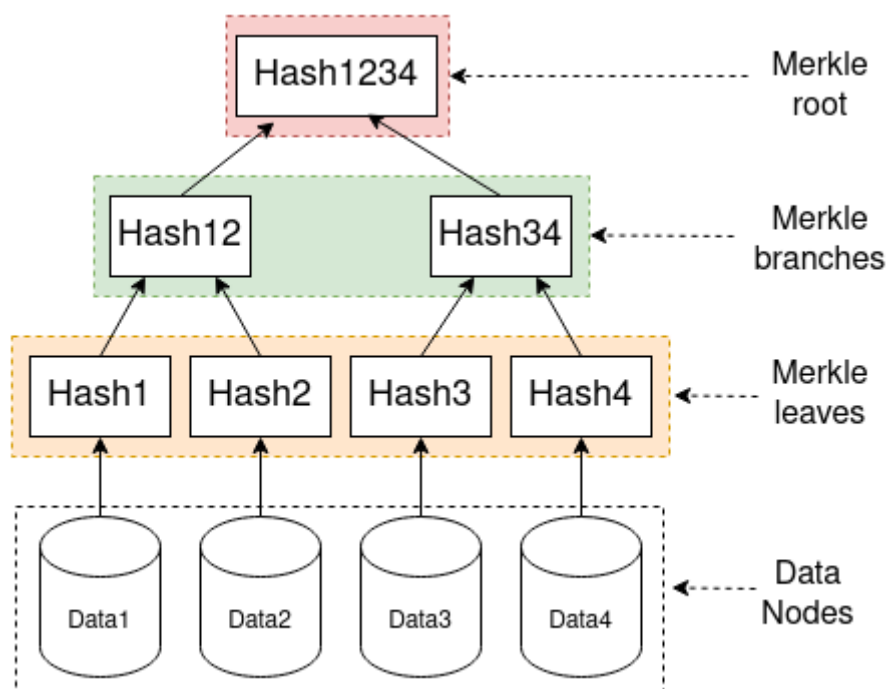
Lo xor rende la $h()$ più resistente alle collisioni:

$$\text{Senza lo xor} \rightarrow H_{i+1} = h(H_i) = E_{m_i}(H_i)$$

Sia $m = m'$ basta trovare H' e tale che $H_{i+1} = E_{m_i}(H_i) = E_{m'_i}(H'_i)$

A quel punto $H' = D_{m'}(E_{m_i}(H_i))$ che è computazionalmente facile.

Merkle Tree



Strategia che permette di determinare in modo molto efficiente l'integrità di un file o del contenuto di un intero disco.

- $h_i = H(B_i)$ con B_i il contenuto del blocco i .

- $h_{ij} = H(h_i || h_j)$ con i e j detti “figli” del nodo ij .

In cima all'albero c'è la radice h_{14} .

Supponendo che il file sia composto da N blocchi, l'albero contiene $2N - 1$ nodi (hash).

Supponiamo che Alice scarichi l'albero da una fonte sicura, quindi inviata in modo autenticato (*firma digitale*).

Verifica di un blocco

Supponiamo che il blocco B_2 sia stato scaricato in modo scorretto o sia stato manomesso. Per verificare l'integrità di $B_2 \rightarrow H(H(H(B_2), h_1), h_{34}) = h_{14}$

Per fare questo controllo serve che Alici scarichi la **lista delle proof**: h_{14} , h_{34} e h_1 .

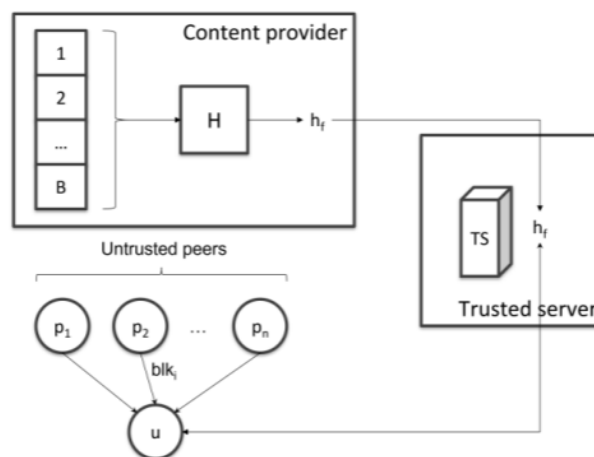
Supponendo che il file sia composto da N blocchi, la verifica ha complessità $\log N$.

Verifica di inclusione di un blocco

L'albero può anche essere usato per determinare se un blocco non è incluso nel file originale. Supponendo che B non appartenga al set di blocchi $\rightarrow B_i < B < B_{i+1}$.

Con B_i e B_{i+1} le foglie adiacenti.

Content distribution



Come fa un peer a verificare che l'informazione ottenuta da un altro peer sia integra o corrotta?

La radice viene inserita in un server sicuro e disponibile ai peers.

Ogni peer memorizza un set di blocchi insieme alla loro proof $\rightarrow \langle blk_i, proof_i \rangle$

Quando un utente scarica un blocco lo verifica usando le $proof_i$ e la radice h_f scaricata dal server sicuro.

MAC

Dobbiamo garantire che un messaggio non sia alterato (**integrità**).

Ma dobbiamo anche garantire la provenienza di un messaggio (**autenticazione**).

Da notare che l'autenticazione dell'origine include l'integrità.

Un sistema MAC include 2 funzioni:

$$S: \{0, 1\}^* \times \{0, 1\}^k \rightarrow \{0, 1\}^m$$

$$V: \{0, 1\}^* \times \{0, 1\}^k \times \{0, 1\}^m \rightarrow \{0, 1\}$$

$t = S_k(x)$ chiamato "**tag**" del messaggio ed è a m bit.

- **Consistenza** $\rightarrow \forall x \forall k, V_k(x, S_k(x)) = 1$

K è la chiave usata per calcolare il MAC, il quale si basa spesso su cifrari simmetrici.

- **Integrità** $\rightarrow$ OK, manipolare il tag o il messaggio porta al fallimento di $V()$.
- **Non-Ripudiabilità** $\rightarrow$ NO, si basa su cifrario simmetrico, quella stessa chiave può anche essere usata altrove; si può ottenere firmando il MAC con una chiave asimmetrica.
- **Autenticazione** $\rightarrow$ OK, producendo il MAC usando K il mittente prova al ricevitore di possedere K .
- **Protezione da replay attack** $\rightarrow$ NO, il MAC non ha cognizione della sessione in cui viene inviato il messaggio.

Attacchi al MAC

In un MAC sicuro non deve poter essere possibile:

- Dato x , generare $t = S_k(x)$ valido senza sapere la chiave K .
 - $S()$ deve essere 2° preimage resistant e strong collision resistant.
 - $S()$ deve essere 1° preimage resistant contro un chosen-text attack.
- Data una coppia $\langle x, t \rangle$, generare una nuova coppia $\langle x', t' \rangle$ valida.
- Data una coppia $\langle x, t \rangle$, generare una nuova coppia $\langle x, t' \rangle$ valida (*ossia trovare una collisione*).

Chosen-message attack

Siano n coppie (x_i, t_i) con $t_i = S_k(x_i)$.

Risulta computazionalmente infattibile calcolare, dato un nuovo messaggio $x \neq x_i \forall i$, un tag $t = S_k(x)$ valido senza sapere la chiave K . Anche se $\exists i : t = t_i$

Forgery Selective

Dato x , l'avversario può generare $\langle x, t \rangle$, $t = S_k(x)$. In questo caso il tag t è creato dal testo stesso.

Forgery Existential

L'avversario può produrre una coppia $\langle x, t \rangle$ valida ma senza avere il controllo sul valore di x .

HMAC

Il HMAC è un MAC costruito partendo da una funzione di hash sicura.

HMAC per autenticazione e confidenzialità

Segui lo schema “*Encrypt then MAC*”.

$$c = E_{K_1}(x)$$

$$t = MAC_{K_2}(y)$$

Questa strategia permette di costruire un MAC che garantisca confidenzialità e autenticazione. Calcolare il MAC direttamente da x può dare informazioni riguardo x .

Attacchi contro HMAC

Attacco al prefisso

$$\text{Prefisso} \rightarrow MAC = H(k || x)$$

$$\text{Suffisso} \rightarrow MAC = H(x || k)$$

$$\text{Sia } x = (x_1, x_2, \dots, x_n)$$

$$\text{Sia } t = S_k(x) = H(k || x_1, x_2, \dots, x_n)$$

Voglio costruire t' da $x' = (x_1, x_2, \dots, x_n, x_{n+1})$ senza sapere k

Supponendo che $H()$ segua lo schema di Merkle-Damgaard $\rightarrow t' = h(x_{n+1}, t)$

Timing Attack

```
def Verify( k , x , t ) :  
  
    return HMAC( k , x ) == t
```

L'operatore "==" esegue un confronto byte per byte, quindi appena trova un byte diverso si interrompe.

Quindi il tempo impiegato dalla funzione varia a seconda di quanti byte giusti ci siano.

Questa debolezza può essere usata per brutare un byte alla volta di x .

Per evitare questo problema basta usare un confronto tra stringhe invece che l'operatore "==".

Oppure si confrontano gli HMAC degli HMAC da confrontare, in modo che l'avversario non conosca i byte che si stanno effettivamente confrontando.

```
def Verify( k , x , t ) :  
  
    m = HMAC( k , x ) ;  
  
    return HMAC( k , m ) == HMAC( k , t )
```

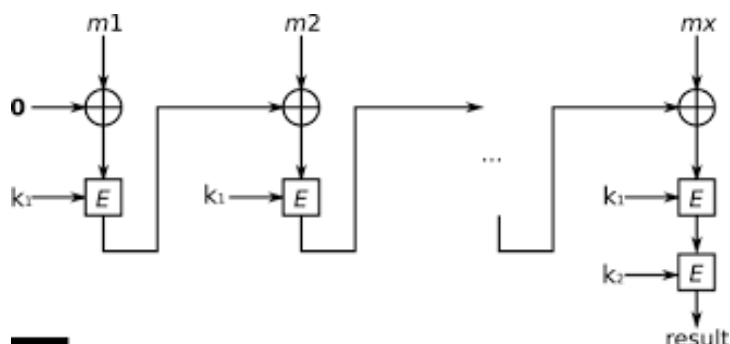
Schema HMAC 1

$$HMAC = h_{IV}([K \oplus opad] || h_{IV}[(K \oplus ipad) || X])$$

$ipad$ e $opad$ hanno valori fissi.

CBC-MAC

Il calcolo di questo MAC non è parallelizzabile.



Alla fine il risultato viene cifrato un'ultima volta usando una chiave diversa $K_1 \neq K$

E_K è un cifrario simmetrico come DES o AES.

La cifratura finale serve perché sennò il sistema è vulnerabile ad un existential forgery attack:

1. L'avversario sceglie un messaggio da un blocco solo.
2. L'avversario richiede il calcolo dell'hash di x .

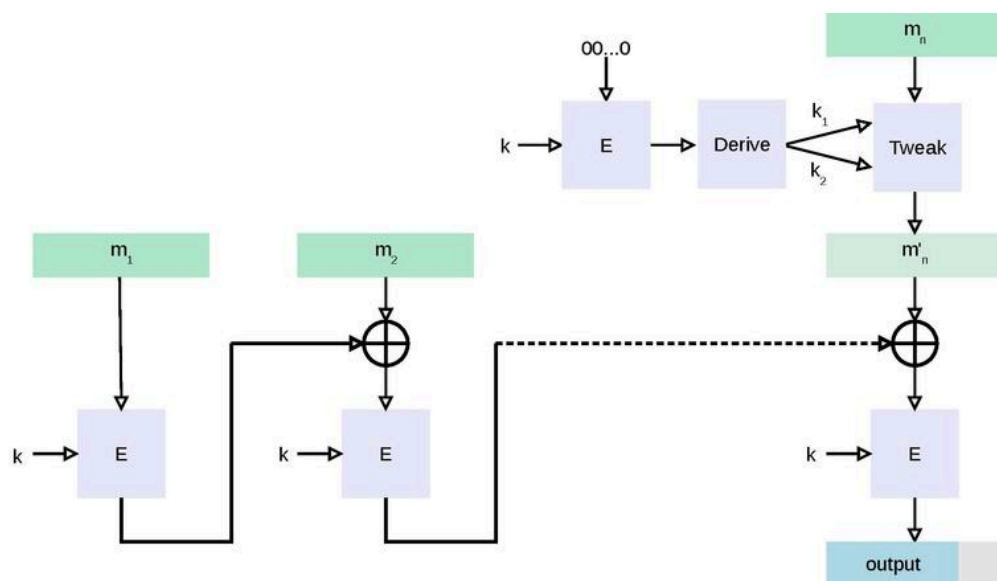
$$t = \text{HMAC}(k, x) = E_k(x)$$

3. L'avversario produce ora un messaggio $x' = \langle x, t \oplus x \rangle$
4. L'avversario richiede il calcolo dell'hash di x' .

$$t' = E(k, x') = E_k((t \oplus x) \oplus E_k(x)) = E_k(t \oplus x \oplus t) = t$$

Si ha lo stesso tag.

CMAC



Risolve i problemi del CBC-MAC.

GMAC

Supponiamo che il crittogramma e i dati associati siano visti come una serie di blocchi

$$X_1, X_2, \dots, X_m$$

$$\text{Sia poi } Y_0 = 0^{128}$$

$$Y_i = (Y_{i-1} \oplus X_i) = (X_1 H^m) \oplus (X_2 H^{m-1}) \dots \oplus (X_m H^1)$$

Con H_i valori pre-calcolati.

Il calcolo di Y_i può essere parallelizzato, ciò rende GMAC molto efficiente.

ECC - Curve Ellittiche

Una curva ellittica è un insieme di punti (x, y) che risolvono l'equazione non lineare del tipo:

$$|y^2|_p = |x^2 + ax + b|_p$$

Una curva ellittica è rappresentata come una curva simmetrica rispetto all'asse delle x.

Sia $Gf(p) = \{0, 1, \dots, p-1\}$

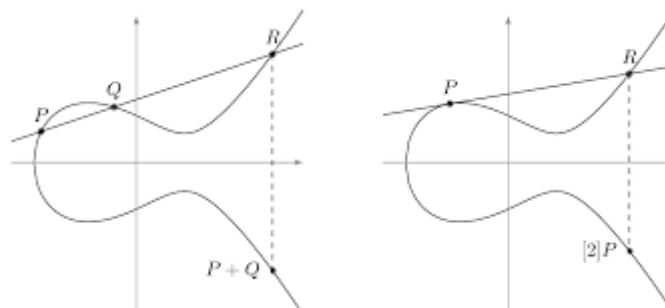
Sia Z_p^* con $p > 3$; $(x, y) \in Z_p^* \Leftrightarrow |y^2|_p = |x^2 + ax + b|_p$

Addizione

Siano questi punti appartenenti ad una curva ellittica:

- $P = (x_1, y_1) \in Z_p^*$
- $Q = (x_2, y_2) \in Z_p^*$
- $R = (x_3, y_3) \in Z_p^*$

Con $R = P + Q$



Quindi:

- $x_3 = |s^2 - x_1 - x_2|_p$

- $y_3 = |s(x_1 - x_3) - y_1|_p$

Con $s = \left| \frac{y_2 - y_1}{x_2 - x_1} \right|_p$ se $P \neq Q$

Con $s = \left| \frac{3x_1^2 - a}{2y_1} \right|_p$ se $P = Q$ (*point doubling*)

Neutrale

$$\forall P \in Z_p^* \exists O : P + O = P$$

$$O \equiv (+\infty, -\infty)$$

Inverso

$$\forall P \in Z_p^* \exists Q : P + Q = O$$

$Q = (-P) = (x, p - y)$ con p il numero associato al modulo della curva ellittica.

Sottogruppi Ciclici

Sia $E \rightarrow |y^2|_{17} = |x^2 + 2x + 2|_{17}$ e sia $P = (5, 1)$

$$2P = (6, 3)$$

$$3P = 2P + P = (10, 6)$$

$$4P = 2P + 2P = 3P + P = (3, 1)$$

...

$$19P = O = \#E * P \rightarrow \text{Neutrale}$$

$$20P = 19P + P = P = (5, 1)$$

$$21P = 20P + P = 19P + 2P = 2P = (6, 3)$$

...

$$\text{Inoltre se } 19P = O \rightarrow 18P + P = O \rightarrow P^{-1} = 18P$$

Th Hasse - Stima del valore di $\#E$

Che valore ha $\#E$? Il teorema di Hasse ci fornisce una stima:

$$\#E \in [p + 1 - 2\sqrt{p}, p + 1 + \sqrt{p}]$$

Con $\#E$ il “*numero di punti*” nella curva E . Calcolare il valore preciso di $\#E$ è molto difficile, per questo ci avvaliamo di questa approssimazione.

La cosa importante di questo teorema: Possiamo stimare il numero di bit necessari per avere un certo grado di sicurezza.

Se voglio $\#E \sim 2^{160} \rightarrow \text{size}(p) \sim 160$ bits.

ECDLP - Logaritmo Discreto su Curve Ellittiche

Data una curva ellittica E , due punti $P, T \in E$; trovare $d \in [1, \#E] : d * P = T$

Questo problema ha complessità $O(\sqrt{\#E})$.

La moltiplicazione, si fa con un algoritmo simile a “square-and-multiply”:

$$26P = (11010)_2 P = (d_4 d_3 d_2 d_1 d_0)_2 P$$

$$\#0a \rightarrow P = (1)_2 P$$

$$\#0b \rightarrow P \rightarrow \text{Ok poiché } d_4 = 1$$

Al punto successivo $\#1a$ si raddoppia ciò che è stato ottenuto in $\#0b$.

$$\#1a \rightarrow P + P = 2P = (10)_2 P$$

$$\#1b \rightarrow 2P + P = 3P = (11)_2 P = (10)_2 P + (1)_2 P \rightarrow \text{Ok poiché } d_3 = 1$$

Al punto successivo $\#2a$ si raddoppia ciò che è stato ottenuto in $\#1b$.

$$\#2a \rightarrow 3P + 3P = 6P = (110)_2 P$$

$$\#2b \rightarrow (110)_2 P = 6P \rightarrow \text{poiché } d_2 = 0$$

Al punto successivo $\#3a$ si raddoppia ciò che è stato ottenuto in $\#2b$.

$$\#3a \rightarrow 6P + 6P = 12P = (1100)_2 P$$

$$\#3b \rightarrow 12P + P = 13P = (1101)_2 P = (1100)_2 P + (1)_2 P \rightarrow \text{Ok poiché } d_1 = 1$$

Al punto successivo $\#4a$ si raddoppia ciò che è stato ottenuto in $\#3b$.

$$\#4a \rightarrow 13P + 13P = 26P = (11010)_2 P$$

$$\#4b \rightarrow (11010)_2 P = 26P \rightarrow \text{poiché } d_0 = 0$$

Regola generale

$\#ia \rightarrow$ Si mette uno 0 (*si moltiplica per 2*) dietro a ciò che è stato calcolato nel punto $\#(i - 1)b$

$\#ib \rightarrow$ Si somma P a ciò che è stato calcolato in $\#ia$ se $d_i = 1$

ECDHKE - DHKE con curve ellittiche

I nostri parametri di dominio sono:

- p un numero primo.
- $E \rightarrow |y^2|_p = |x^2 + ax + b|_p$ una curva ellittica, quindi (a, b) .
- Un punto $P \in E$ primitivo.

Algoritmo:

1. Alice sceglie $S_{KA} = a \in \{2, 3, \dots, \#E - 1\}$ e calcola $P_{KA} = aP = A$
2. Bob sceglie $S_{KB} = b \in \{2, 3, \dots, \#E - 1\}$ e calcola $P_{KB} = bP = B$
3. Alice invia a Bob

M1 $A \rightarrow B$: "*Alice*", A

4. Bob invia ad Alice:

M2 $B \rightarrow A$: "*Bob*", B

5. Alice calcola $T_{AB} = a * B = a * b * P$
6. Bob calcola $T_{AB} = b * A = b * a * P$

$T_{AB} = (x_{AB}, y_{AB}) \in E$; nota che x_{AB} e y_{AB} non sono indipendenti tra loro.

7. $K_{sess} = |H(x_{AB})|_{256}$

Attacchi al ECDHKE

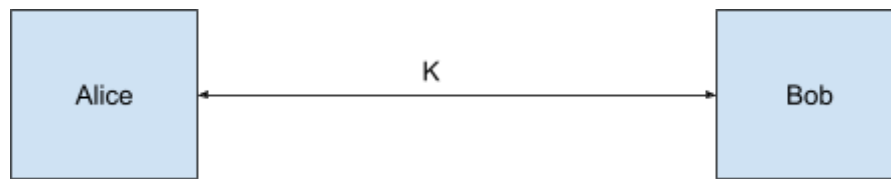
L'avversario vede: A e B ; deve trovare a o b .

L'unico modo per trovare uno dei due è risolvere il problema del logaritmo discreto su curve ellittiche ECDLP il quale ha complessità $O(\sqrt{\#E})$.

Se voglio $\#E \sim 2^{160}$ quindi 80 bit di sicurezza (*sicurezza di medio livello*), ho bisogno di $size(p) \sim 160$ bit.

Nelle curve ellittiche se $size(\#E) = n \rightarrow \frac{n}{2}$ bit di sicurezza. Di solito si usa $\#E \sim 2^{256} \rightarrow 128$ bit di sicurezza.

Session key establishment



Alice e Bob condividono un segreto a lungo termine K_{AB} , ma per comunicare usano un segreto a breve termine K ottenuto partendo dal segreto a lungo termine.

Vantaggi della chiave di sessione

- Se K è compromessa, solo una parte dei messaggi scambiati saranno compromessi: K_{AB} e le altre chiavi di sessione rimangono segrete.
- K è usata per poco tempo $\rightarrow$ Meno messaggi utilizzabili per attacchi analitici.
- Un avversario, può essere obbligato a recuperare molteplici K per ricostruire un singolo messaggio.

One-pass Key Transport

Modalità più semplice per effettuare uno scambio di chiave di sessione.

$$M1 \quad A \rightarrow B: \quad E_{K_{AB}}(K_{sess} || t_A)$$

t_A un timestamp per garantire la freschezza della chiave; il timestamp può dare problemi riguardo la sincronizzazione del clock tra A e B.

Challenge-Response Key Transport

Nel caso si può usare anche un nonce.

$$M1 \quad B \rightarrow A: \quad n_B$$

$$M2 \quad A \rightarrow B: \quad E_{K_{AB}}(K_{sess} || n_B)$$

n_B un nonce per garantire la freschezza della chiave.

Key Agreement

M1 $B \rightarrow A: n_B$

M2 $A \rightarrow B: E_{K_{AB}}(K' || n_B || n_A)$

M3 $B \rightarrow A: E_{K_{AB}}(K'' || n_A)$

A e B creano la vera $K_{sess} = H(K' || K'')$.

Key distribution problem

Ho n utenti e tutti vogliono comunicare con tutti. Ogni coppia di utenti ha una chiave a lungo termine K_{AB}

In tutto ci sono $N_c = \frac{n(n-1)}{2}$ coppie $\rightarrow N_c \sim O(n^2)$

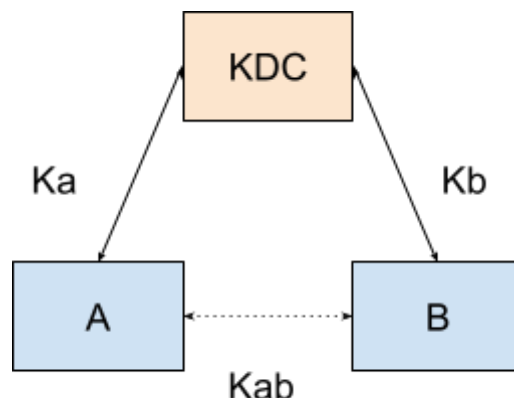
Sicurezza ottima: Se una di queste chiavi è compromessa, le altre non lo sono.

Scalabilità pessima: Ogni utente deve tenere memorizzare $(n - 1)$ chiavi a lungo termine. Inoltre, l'aggiunta/rimozione di un utente causa la creazione/cancellazione di chiavi da parte degli altri utenti.

Key Distribution Center

Soluzione al key distribution problem.

Istituisco un componente speciale chiamato “KDC”, il suo compito è quello di creare le chiavi di sessione per una coppia di utenti.



Ogni utente ha una chiave a lungo termine SOLO con il KDC, quindi invece che $\frac{n(n-1)}{2}$ chiavi a lungo termine ne ho solo n .

1. Alice richiede al KDC di comunicare con Bob

"req", "Alice", "Bob"

2. KDC calcola:

- $K_{AB} = rand()$
- $y_A = E_{KA}(K_{AB})$ e $y_B = E_{KB}(K_{AB})$

3. A quel punto il KDC:

- **1° Versione:**
 - i. Invia y_A ad Alice e invia y_B a Bob
- **2° Versione:**
 - i. Invia y_A e y_B ad Alice.
 - ii. Alice invia y_B a Bob.

4. Alice calcola $K_{AB} = D_{KA}(y_A)$

5. Bob calcola $K_{AB} = D_{KB}(y_B)$

Questo algoritmo è vulnerabile al MIM

Un avversario può mettersi tra Alice e il KDC.

1. Alice richiede al KDC di comunicare con Bob.

"req", "Alice", "Bob"

2. Il MIM intercetta la richiesta, la modifica e invia al KDC

"req", "Alice", "MIM"

3. KDC calcola

- $K_{AC} = rand()$
- $y_A = E_{KA}(K_{AC})$
- $y_C = E_{KC}(K_{AC})$

4. KDC Invia y_A e y_C ad Alice.

5. Alice invia y_C al MIM.
6. Alice calcola $K_{AC} = D_{KA}(y_A)$
7. MIM calcola $K_{AC} = D_{KC}(y_C)$

Perfect Forward Secrecy

Compromettere K_{AB} non compromette la segretezza delle chiavi di sessione scambiate in precedenza.

PKE-based Key Exchange

Supponendo che Alice abbia il certificato di Bob e che sia valido.

1. Alice calcola $K = rand()$ e invia a Bob:

M1 $A \rightarrow B: \quad "Alice", E_{P_{KB}}(K)$

2. Bob calcola $K = D_{S_{KB}}(K)$

In questo caso S_{KB} è il segreto a lungo termine; ma la PFS non è garantita: se S_{KB} è compromessa allora l'avversario può ricavare tutte le chiavi usate prima.

Ephemeral Diffie-Hellman (PSK-DKE)

Supponendo che esista già il segreto a lungo termine K_{AB} tra Bob e Alice.

1. Alice calcola $a = rand()$ e invia a Bob

M1 $A \rightarrow B: \quad "Alice", E_{K_{AB}}(|g^a|_p)$

2. Bob calcola $b = rand()$ e invia ad Alice

M2 $B \rightarrow A: \quad "Bob", E_{K_{AB}}(|g^b|_p)$

3. Alice calcola $K = |g^{ba}|_p$

4. Bob calcola $K = |g^{ba}|_p$

a e b sono effimeri e utilizzabili solo una volta e poi devono essere cancellati, altrimenti un avversario può ricavare K .

La PFS è garantita? Sì. Se K_{AB} è compromessa, K non lo è, poiché l'avversario deve comunque ricavare a e b per i quali serve risolvere un DLP.

RSAE - Ephemeral RSA

Supponendo che esista già il segreto a lungo termine K_{AB} .

1. Alice calcola $R = rand()$ e invia a Bob

M1 $A \rightarrow B$: "Alice", R

2. Bob genera due chiavi RSA **temporanee** $\langle T_{P_{KB}}, T_{S_{KB}} \rangle$

3. Bob invia ad Alice:

M2 $B \rightarrow A$: $T_{P_{KB}}, \langle R || T_{P_{KB}} \rangle_{S_{KB}}, Cert_B$

P_{KB} e S_{KB} le chiavi RSA a lungo termine di Bob.

$Cert_B$ il certificato della chiave RSA a lungo termine di Bob.

4. Alice calcola $K_{sess} = rand()$ e invia a Bob.

Alice per verificare la firma usa la chiave pubblica di Bob estratta da $Cert_B$.

M3 $A \rightarrow B$: $E_{T_{P_{KB}}}(K)$

5. Bob calcola $K_{sess} = D_{T_{S_{KB}}}(E_{T_{P_{KB}}}(K))$.

Alla fine del protocollo Bob cancella $\langle T_{P_{KB}}, T_{S_{KB}} \rangle$

Direct Authentication

Prova all'interlocutore la conoscenza di un segreto a lungo termine x .

Se il protocollo di scambio delle chiavi non soddisfa questa cosa, allora deve essere fatto a livello applicativo (*tramite una password*).

Station-to-Station Protocol

Soddisfa la direct authentication.

1. Alice calcola $a = rand()$ e invia a Bob

M1 $A \rightarrow B$: "Alice", g^a

2. Bob calcola $b = rand()$ e calcola $K_{DH} = |g^{ab}|_p$.

3. Bob invia ad Alice.

M2 $B \rightarrow A$: "*Bob*", g^b , $\{<g^b, g^a>_{B^K}, Cert_B$

4. Alice invia a Bob, per provare che conosce K_{DH}

M3 $A \rightarrow B$: "*Alice*", $\{<g^b, g^a>_{A^K}, Cert_A$

5. $K = KDF(K_{DH}, \dots)$

Alla fine a e b devono essere cancellati.

Kerberos

Domanda generica su kerberos: a cosa serve e come funziona?

Si basa sul protocollo di Needham-Schroeder. gli obiettivi di kerberos sono:

- **Sicurezza:** Rendere impossibili lo spoofing e lo eavesdropping per un esterno all'organizzazione.
- **Disponibilità:** Se kerberos è non disponibile → Tutti i servizi diventano non disponibili.
- **Replicabile:** Per renderlo altamente replicabile, kerberos deve essere stateless, quindi non deve tenere traccia di connessioni passate o altro.
 - Il database delle password deve essere a sua volta replicabile.
- **Trasparenza:** Il processo di autenticazione di un utente deve avvenire tramite password.
- **Scalabilità:** Kerberos deve poter gestire un elevato numero di utenti e servizi.

Supponiamo che ci sia un client (*Alice, utente registrato nell'organizzazione*) che vuole usufruire di un servizio offerto da un server (*Bob*).

Supponiamo anche che la macchina client sia “*stupida*”, ossia che non memorizzi nulla in locale.

Quindi, concretamente i nostri obiettivi sono:

- **Autenticazione mutuale:** Il client vuole autenticare il server e viceversa.
- **Chiave di sessione:** Stabilire una chiave di sessione tra il client e il server.

Kerberos si avvale di due dispositivi:

- **AS** → Authentication System
- **TGS** → Ticket Granting System

Protocollo senza delegazione

1. Il client "Alice" invia le sue credenziali (*username e password*) allo AS per autenticarsi.

"Supponiamo che il login sia avvenuto con successo"

2. Il client invia anche una richiesta per comunicare con il server "Bob".

M1 $A \rightarrow AS: \quad "Alice", "Bob", t, L, N_a, WS$

- $WS \rightarrow$ Workstation da cui Alice ha fatto il login.
- $t, L \rightarrow$ Durata di validità del TGS, di solito $L \sim$ "qualche ora".

3. Il AS invia al client un TGT (*Ticket Granting Ticket*) da inoltrare al TGS.

M2 $AS \rightarrow A: \quad \{"Alice", TGS, t, L, T_K, WS\}_{K_{TGS}}$

- $\{"Alice", TGS, t, L, T_K, WS\}_{K_{TGS}} \rightarrow TGT$
 - $t, L \rightarrow$ Durata del TGT.
 - $K_{TGS} \rightarrow$ Chiave che conosce solo il TGS e lo AS.
 - $T_K \rightarrow$ Chiave tra Alice e il TGS

4. Il client invia il TGT al TGS

M3 $A \rightarrow TGS:$

$\{"Alice", t_a\}_{T_K}, ("Bob", t', L', N_a), \{"Alice", TGS, t, L, T_K, WS\}_{K_{TGS}}$

- $\{"Alice", t_a\}_{T_K} \rightarrow Auth_A \rightarrow$ Authenticator di Alice.
- $("Bob", t', L', N_a) \rightarrow$ Informazioni da mettere nel ticket per parlare con il server.
- $\{"Alice", TGS, t, L, T_K, WS\}_{K_{TGS}} \rightarrow TGT$

5. Il TGS invia al client un ticket ST_B (*service ticket*).

M4 $TGS \rightarrow A:$

$\{"Alice", "Bob", t', L', K_{AB}, WS\}_{K_B}, \{"Bob", t', L', K_{AB}, WS, N_a\}_{T_K}$

- $\{"Alice", "Bob", t', L', K_{AB}, WS\}_{K_B} \rightarrow ST_B \rightarrow$ Service ticket da inoltrare a Bob.

- $\{ "Bob", t', L', K_{AB}, WS, N_{aT_K} \} \rightarrow ST_A \rightarrow$ Service ticket che Alice deve tenere.

6. Il client inoltra a Bob il ST_B

M5 $A \rightarrow B:$ ST_B

7. Bob controlla la validità del ticket.

Il ticket ST_B è considerato valido dal server se $|t_b - t'| \leq \lambda$.

- λ è detto “*authenticator lifetime*”; deve avere un valore consono, poiché deve adattarsi alla desincronizzazione del clock tra server e client; quindi di solito $\lambda \sim 5 \text{ min}$.
- t_b è il timestamp corrente misurato da Bob al momento dell’arrivo del messaggio M5.
- t' è il timestamp presente dentro M5.

Protocollo con delegazione

1. Il client fa richiesta allo AS per comunicare con il mail server.

Da notare che il mail server si avvale dei servizi di un file system distribuito (consideriamolo come un altro server), quindi il mail server deve accedere ai servizi di questo server per conto di Alice; quindi sfruttando il livello di privilegio minimo per fare il proprio lavoro.

2. Il AS invia al client un FTGT (Forwardable Ticket Granting Ticket).
3. Il client invia il FTGT al TGS.
4. Il TGS invia al client un ticket proxy ST_{MS} e un TGT_{MS} (service ticket).

$$ST_{MS}: \{ \dots, K_{AMS} \dots \}_{T_K}, \{ \dots, K_{AMS}, \dots \}_{K_{MS}}$$

$$TGT_{MS}: \{ \dots, T'_K, \dots \}_{T_K}, \{ \dots, T'_K, "MS", \dots \}_{K_{TGS}}$$

5. Il client inoltra ST_{MS} e il TGT_{MS} al MS, tutto cifrato con K_{AMS} .
6. Il MS invia TGT_{MS} al TGS, ossia una richiesta per parlare con il FS.
 - Questo messaggio è cifrato con la chiave T_K in modo che il TGS sappia che il MS sta parlando per conto di Alice.
7. Il TGS invia allo MS un ticket per parlare con il FS.
8. Il MS invia al ticket al FS come in una normale comunicazione client-server.

$$ProxyTicket: \{ K_{AFS}, N_a, t, L, "FS" \}_{T_K}, \{ K_{AFS}, "MS", s, L \}_{K_{fs}}$$

- $\{ K_{AFS}, "MS", s, L \}_{K_{fs}} \rightarrow$ Il FS è sicuro dell'autenticità di questo messaggio perché K_{fs} è conosciuta solo da lui e dal TGS.

i pro di questo approccio:

- L'utente controlla tutti i diritti che delega al server.
- Grazie al forwardable ticket, è il server che decide quali ticket gli servono.

Il problema di questo approccio è che:

- Il client deve sapere a priori tutti i ticket necessari.
- Un server compromesso può abusare di tutti i diritti.

Protocollo Needham-Schroeder

Altro

One-Time Passwords

La generazione di queste password può basarsi su 3 principi:

Sincronizzazione temporale

1. Il prover genera $t_p = \text{currentClock}()$
2. Il prover calcola $HOTP = \text{HMAC}_K(\frac{t_p - t_0}{X})$ e lo invia all'authenticator.
3. L'authenticator genera $t_v = \text{currentClock}()$
4. $\forall t \in [t_v - \frac{W}{2}, t_v + \frac{W}{2}] \rightarrow$ Il for server per evitare errori dovuti a sincronizzazione.
 - a. $C_v = \frac{t - t_0}{X}$
 - b. Se $HOTP == \text{HMAC}_k(C_v) \rightarrow$ L'authenticator invia 1 al prover.
 - c. return.
5. Altrimenti $\rightarrow$ L'authenticator invia 0 al prover.

Passwords precedenti - Schema di Lamport

Setup:

1. Assegno al seed $p_0 = \text{rand}()$ una stringa casuale.
2. $p_i = H(p_{i-1}) \forall i \in [1, n]$
3. p_n è memorizzata localmente dal verifier.

Verifica:

1. Prover invia p_{n-1} al verifier.
2. Il verifier verifica se una di queste due condizioni risulta vera:
 - $p_n = H(p_{n-1})$
 - $p_n = H^i(p_0) = H(H(\dots H(p_0)\dots)) \rightarrow$ Le password non sono verificate in modo sequenziale, ma si riparte sempre dall'inizio.
3. Se si ritorna invia "1" al prover.

Challenge-response

Supponendo che client e server condividano una chiave K a priori.

1. Il verifier genera $\text{challenge} = \text{rand}()$ e la invia al prover.

2. Il prover calcola $R = \text{HMAC}_K(\text{challenge})$ e la invia al verifier.
3. Il verifier verifica se $R = \text{HMAC}_K(\text{challenge})$

Password storage

Vanno memorizzate in questo formato:

Username	Salt	Hash
----------	------	------

Con $\text{hash} = H(\text{password} || \text{salt})$

L'aggiunta del salt rende un rainbow attack infattibile.

Se il digest della password è contenuto in un altro file, rende anche un dictionary attack infattibile.

Rainbow Attack

Supponiamo che le password siano conservate in questo modo:

Username	Hash
----------	------

Con $\text{hash} = H(\text{password})$

Un Rainbow Attack sfrutta tabelle precompilate di hash per craccare password.

Queste tabelle, chiamate "tabelle arcobaleno", contengono hash di password comuni e combinazioni di caratteri insieme alle relative password originali.

Ecco come funziona in dettaglio:

1. **Creazione delle Tabelle Arcobaleno:** Gli attaccanti precompilano una tabella che associa hash a password originali.
 - a. Questa tabella può contenere milioni o miliardi di hash per password comuni e combinazioni di caratteri.
2. **Confronto degli Hash:** Quando l'attaccante ottiene un hash (*ad esempio, da un database violato*), cerca questo hash nella tabella arcobaleno.
3. **Recupero della Password:** Se l'hash è presente nella tabella, l'attaccante può trovare la password originale associata a quell'hash poiché contenuta nella tabella.

Bit di sicurezza

Vediamo quanti bit di chiave sono necessari per ottenere un certo livello di sicurezza.

Bit di sicurezza voluti	80	128	192	256
RSA	1024	3072	7680	15360
Diffie-Hellman , DSA	1024	3072	7680	15360
Curve Ellittiche (ECC)	160	256	384	512
Chiave simmetrica	80	128	192	256

Esercizi

Esercizio su CTR e CBC

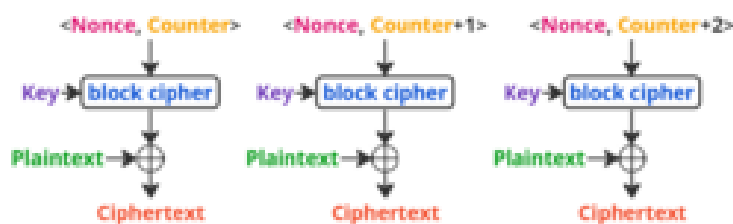
Alice and Bob have to choose an encryption mode between CTR and CBC (*both modes require an IV that is transmitted in the clear*).

They know the adversary is able to intercept and change messages exchanged between them.

They also know that for some messages the adversary knows the first plaintext block M_1 and wants to replace it with another block of his choice, namely A_1 .

1. Does CTR make it possible to perform such an attack?
2. Does CBC make it possible to perform such an attack

CTR



Riguardo la modalità CTR l'attacco è possibile: Sia

- $C_0 = M_0 \oplus Z_0$ dove $Z_0 = E_K(IV)$
- $C_1 = M_1 \oplus Z_1$ dove $Z_1 = E_K(IV + 1)$

L'attacco si svolge così:

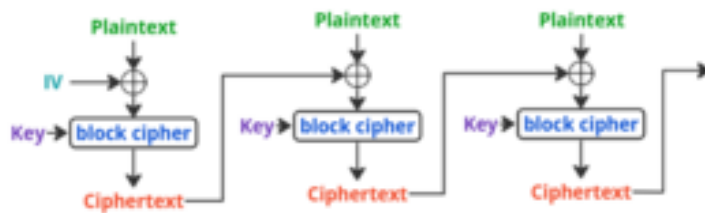
1. L'avversario intercetta il messaggio (IV, C_1)

L'avversario conosce la corrispondenza $M_1 \leftrightarrow C_1$ dato un certo IV .

2. L'avversario quindi calcola $Z_1 = M_1 \oplus C_1$

3. L'avversario poi calcola $C_1' = A_1 \oplus Z_1 = (A_1 \oplus M_1) \oplus C_1$ e lo rimpiazza.

CBC



L'attacco è possibile.

- $C_1 = E_K(M_1 \oplus IV)$
- $C_2 = E_K(M_2 \oplus C_1)$
- $C_3 = E_K(M_3 \oplus C_2)$

L'avversario conosce IV , C_1 , C_2 , C_3 , M_1 e conosce che $M_1 \leftrightarrow C_1$

1. L'avversario sa che $M_1 = D_K(C_1) \oplus IV$
2. Quindi calcola $IV' = M_1 \oplus IV \oplus A_1$
3. L'avversario rimpiazza IV con IV' , nota che C_1 NON viene alterato.
4. Quando la vittima decifra $\rightarrow D_K(C_1) \oplus IV' = A_1$

Esercizio sicurezza protocolli 1

In an electronic auction, bidder Bob casts his bid B encrypting it by means of auctioneer Alice's public key P_{KA} .

Let us assume that B is a 32-bit unsigned.

A protocol is secure if the guessing attack requires at least $2^{(80)}$ steps.

1. $B \rightarrow A$: $"Bob", \{"Bob", B\}_{P_{KA}}$
2. $B \rightarrow A$: $"Bob", \{"Bob", B, H(B)\}_{P_{KA}}$
3. $B \rightarrow A$: $"Bob", \{"Bob", H(B)\}_{P_{KA}}$
4. $B \rightarrow A$: $"Bob", R, \{"Bob", R, B\}_{P_{KA}}$

5. $B \rightarrow A$: $"Bob", \{ "Bob", R, B \}_{P_{KA}}$
6. $B \rightarrow A$: $"Bob", \{ "Bob", K \}_{P_{KA}}, \{ "Bob", B \}_K$

In the protocols,

- $\{x\}_K$ denotes the encryption of x under key K .
- $H()$ is a secure hash function whose output size is h -bit.
- R is an r -bit random number,
- K is a k -bit random symmetric cryptographic key.
- R and K are generated dynamically at bidding time.

Select parameters h, r and k so that secure protocols have at least 80-bit security level.

Argue whether the following protocols 1-6 below are practical and secure w.r.t. to a passive adversary who attempts to guess the bid B .

1. Il primo protocollo è insicuro, un numero a 32 bit è molto piccolo, quindi avendo la P_{KA} si può fare un attacco esaustivo sullo spazio dei 32 bit di B , dato che il crittogramma è deterministico, appena trovo un crittogramma uguale ho trovato anche B .
2. Come nel caso 1, la funzione $H()$ è efficiente, quindi basta fare un attacco esaustivo su B adattando $H(B)$ ad ogni tentativo.
3. Come nel caso 2; inoltre è inutile, perché l'auctioner deve indovinare B quindi tremendamente inefficiente.
4. Insicuro per lo stesso motivo del caso 2, dato che R è inviato in chiaro.
5. Supponendo che R abbia abbastanza bit è sicuro. Basta che R abbia almeno 48 bit.
 - $r \geq 48$
6. Sicuro, l'avversario deve indovinare K , la quale se di almeno 80 bit rende il protocollo sicuro.
 - $k \geq 80$

Argue the case the bid B is not uniformly distributed but falls in the interval $[B_1, B_2]$, with B_1, B_2 unsigned integer and $B_1 < B_2$.

1. Il primo protocollo è ancora più insicuro dato che lo spazio di B è diminuito.
2. Ancora più insicuro, la funzione $H()$ è efficiente, quindi basta fare un attacco esaustivo su B adattando $H(B)$ ad ogni tentativo.
3. Ancora più insicuro come nel caso 2; inoltre è inutile, perché l'auctioner deve indovinare B quindi tremendamente inefficiente.
4. Ancora più insicuro per lo stesso motivo del caso 2, dato che R è inviato in chiaro.

5. Supponendo che R abbia abbastanza bit è sicuro. Ma stavolta, R dovrà avere più bit per compensare il difetto di B.
 - Supponendo che lo spazio di B ora sia di $p \leq 32$ bit.
 - $r \geq 80 - p$
6. Dato che l'unico attacco è fatto sulla chiave questo protocollo rimane sicuro.
 - $k \geq 80$

Esercizio session key

Alice and Bob maintain a trust relationship with Trent, a trusted third party, that materializes in the shared secret keys K_A and K_B .

Design a protocol that makes it possible to establish a session key K_{AB} between Alice and Bob and that satisfies the following requirements:

- The protocol guarantees perfect forward secrecy;
- The protocol is robust w.r.t. replay attacks;
- The protocol guarantees key authentication and key confirmation;

Assume that clocks are synchronized.

M1 $A \rightarrow T :$ $E_{K_A}(A || B || y_A || t_A)$

- con $y_A = rand()$
- con $t_{A1} = currentTimestamp$

M2 $T \rightarrow B :$ $E_{K_B}(A || B || t_{c1} || y_A)$

- con $t_{c1} = currentTimestamp$

M3 $B \rightarrow T :$ $E_{K_B}(A || B || t_B || y_B)$

- con $t_{B1} = currentTimestamp$
- con $y_B = rand()$

M4 $T \rightarrow A :$ $E_{K_A}(A || B || t_{c2} || y_B)$

- con $t_{c2} = currentTimestamp$

M5 $A \rightarrow B :$ $E_{K_{AB}}(A || B || t_{A2}) \rightarrow \text{Key Confirmation}$

- con $t_{A2} = currentTimestamp$

M6 $B \rightarrow A :$ $E_{K_{AB}}(A || B || t_{B2}) \rightarrow \text{Key Confirmation}$

- con $t_{B2} = \text{currentTimestamp}$

Garantisce la PFS? Sì, poiché la compromissione di K_{AB} non comporta la compromissione di chiavi di sessione successive o precedenti.

L'inserimento dei timestamp permette di riconoscere messaggi ripetuti e quindi previene dai replay attacks.

Esercizio identification protocol 1

Consider the following identification protocol that allows Alice to identify itself w.r.t. Bob where

- c is a 128-bit random challenge.
- r is the corresponding response.
- v is a 24-bit random initialization vector.

M1 $A \rightarrow B:$ Identification Request

M2 $B \rightarrow A:$ c

M3 $A \rightarrow B:$ v, r

M4 $B \rightarrow A:$ 1 oppure 0

Upon receiving message M2 carrying a challenge c from Bob, Alice generates an initialization vector v at random.

Alice computes the response r by encrypting c as follows:

$$r = \text{SPRG}(k || v) \oplus c$$

- $\text{SPRG}(k || v)$ è una sequenza a 128 bit generata da un PRG.
- k is a long-term cryptographic key a-priori secretly shared by Alice and Bob.

Upon receiving r and v , Bob computes $r' = \text{SPRG}(k || v) \oplus c$ and returns ($r' == r$) to Alice.

Does this protocol guarantee identification? Can a passive adversary impersonate Alice?

Un avversario vede (*eavesdropping*): c, r, v

Quindi può calcolare $\rightarrow z = \text{SPRG}(k || v) = r \oplus c$

L'avversario usando la coppia (v, z) può identificarsi quando vuole, poiché è sempre valida.

Let us suppose now that Bob sends Alice an initialization vector v together with the challenge c in message M2 which becomes $\langle c, v \rangle$.

M1 A $\rightarrow$ B: Identification Request

M2 B $\rightarrow$ A: c, v

M3 A $\rightarrow$ B: r

M4 B $\rightarrow$ A: 1 oppure 0

Define a dictionary attack against this variant of the protocol and evaluate the size in bytes of the dictionary.

L'avversario non ha più il controllo su v e quindi non può usare la strategia di prima. Per eseguire un dictionary attack l'avversario deve collezionare molte coppie $\langle (c, v), r \rangle$

- $(c, v) \rightarrow 152$ bit.
- $r \rightarrow *$ bit.

In totale un dizionario completo avrebbe 2^{152} righe.

Ma in realtà ne bastano molte meno, poiché comunque $SPRG(k || v)$ dipende ancora da k (che è fissa) e da v . Quindi basta creare un dizionario solo su v .

In totale il dizionario completo avrà $2^{24} \sim 4'000'000$ di righe, che rende questo attacco decisamente fattibile.

Ogni riga è del tipo $\langle v, SPRG(k || v) \rangle$ e peserebbe $(24 + 128) = 152$ bit, ovvero 19 byte per riga.

In totale il dizionario peserebbe 302,81 megabytes.

Esercizio sicurezza di una hash function 1

Let $h: \{0, 1\}^* \rightarrow \{0, 1\}^n$ be a hash function that is second-preimage and collision resistant.

Let us now consider the hash function $h': \{0, 1\}^* \rightarrow \{0, 1\}^{n+1}$ built from $h()$ as follows:

- $h'() = 0 || x$ se $x < 2^n$
- $h'() = 1 || h(x)$ se $x \geq 2^n$

Argue whether $h'()$ is pre-image, second-preimage, and collision resistant.

Pre-image resistance

Dato un output di $h()$, $y \in \{0, 1\}^n$ deve essere computazionalmente infattibile trovare $x : y = h(x)$.

- Se y comincia per 0, allora so già che $x < 2^n$ e quindi basta rimuovere il prefisso di 0 (*complessità computazionale nulla*).
- Se y comincia per 1, allora so già che $x \geq 2^n$ e quindi basta eseguire un forgery attack sugli altri n bit come avrei fatto su $h()$.

$h'()$ non è pre-image resistant.

2° Pre-image resistance

Dato $y = h(x)$ deve essere computazionalmente infattibile trovare $x' \neq x : y = h(x')$.

- Se y comincia per 0, allora non esiste una collisione per $x < 2^n$.
- Se y comincia per 1, allora occorre fare un attacco su $h()$ la quale soddisfa la second-preimage resistant.

$h'()$ è second-pre-image resistant.

collision resistance

Dato x deve essere computazionalmente infattibile trovare $x' : h(x) = h(x')$

Questa proprietà implica anche la 2° preimage resistance ma non implica la 1° preimage resistance

- Se $x < 2^n$ non ci può essere una collisione.
- Se $x \geq 2^n$ allora occorre fare un attacco su $h()$ la quale è collisione resistant.

Esercizio sulla sicurezza di uno stream cipher 1

Consider the following methods for integrity protection with encryption.

In both cases, the cipher E is a stream cipher.

Method 1 $\rightarrow c = E_K(x || h(x))$

Show that, if the attacker knows the whole plaintext x , he can replace x by an arbitrary x' of his choice and compute c' such that the receiver will verify the message correctly.

Assume that x and x' are of equal length.

1. L'avversario conosce x , quindi calcola $h(x)$
2. Sapendo c , esegue un attacco esaustivo per trovare k .
3. Una volta trovata k può calcolare e inviare $c' = E_K(x', h(x'))$

Will this attack work too if the encryption is done with a one-time pad $\rightarrow$

$$c = x \oplus k$$

Con OTP funzionerebbe comunque, dato che in questo attacco la funzione E_K è vista come una black-box.

Method 2.

Is the attack still applicable if the checksum is computed using a MAC:

$$c = E_{k_1}(x || HMAC_{k_2}(x))$$

1. L'avversario conosce x , ma non può calcolare $HMAC_{k_2}(x)$ senza k_2 .
2. Sapendo c , dovrebbe eseguire un attacco esaustivo su x e su k_2 per trovare k_1 .
3. Una volta trovata la combinazione k_1, k_2 giuste può calcolare e inviare c'

Ma a differenza di prima, il punto 2 richiede uno sforzo enormemente più grande, dato che (*supponendo x e k_2 a n bit, il livello di sicurezza è raddoppiato*).

Esercizio identification protocol 2

For identification, Alice and Bob share a PIN composed of $d = 4$ digits, and perform the following challenge-response protocol:

1. M1 A $\rightarrow$ B: *challenge*
2. M2 B $\rightarrow$ A: *response*

Indicate which one of the following implementations of CHL and RSP is secure w.r.t. an off-line password-guessing attack.

For each one, estimate the attack complexity.

1. $chl = r_a$ and $rsp = E_K(r_a)$
2. $chl = r_a$ and $rsp = E_K(r_a || r_b)$
3. $chl = r_a$ and $rsp = HMAC_K(r_a || r_b)$
4. $chl = r_a$ and $rsp = E_{P_{KA}}(t), t = HMAC_K(r_a)$

$$5. chl = r_a \text{ and } rsp = E_{P_{KA}}(t || r_b), t = HMAC_K(r_a)$$

where:

- r_a and r_b are nonces generated by Alice and Bob, respectively;
- K is a symmetric key, with $K = h(PIN)$, where $h()$ is a secure hash function; and,
- P_{KA} is A's public key, known to B.

Quindi:

Il PIN ha 4 cifre e quindi 10'000 valori possibili; per cui il pin sta su 14 bit.

1. L'avversario conosce chl e rsp , per trovare K basta fare un attacco esaustivo sulle cifre del pin per indovinare K .
 - Per capire se K è giusta confronta r_a con il contenuto decifrato.
 - 2^{14} valutazioni della funzione $E()$ e $h()$.

Supponendo r_a su m_a bit e r_b su m_b bit.

2. Per trovare K basta fare un attacco esaustivo sulle cifre del pin per indovinare K .
 - Per capire se K è giusta confronta r_a con il primo campo del contenuto decifrato.
 - Quindi 2^{14} valutazioni della funzione $E()$ e $h()$.
3. Insicuro per lo stesso motivo del punto (2), inoltre il ricevitore non potrebbe verificarne il contenuto dato che non conosce r_b .
4. Insicuro, l'avversario compie un attacco esaustivo su k , poi per capire se ha trovato la password, cifra ciò che ha ottenuto con la P_{KA} , se ciò che ha ottenuto alla fine è uguale a res , allora sono ha trovato K .
5. Sicuro, l'avversario non può cifrare con P_{KA} perché non conosce t_b

Esercizio sulla sicurezza di una firma 1

Given the two protocols P1 and P2 with

1. $E_{K1}(x || HMAC_{K2}(x))$ where $E()$ is a symmetric cipher.
2. $x, E_P(H(x))$ dove $E()$ is an asymmetric cipher e P_K è la chiave pubblica.

State whether they, respectively, satisfy the following security services:

confidentiality

Il primo protocollo garantisce confidenzialità poiché x è inviato cifrato dentro al crittogramma.

Il secondo protocollo non garantisce confidenzialità, x è inviato in chiaro.

integrity

Il primo protocollo garantisce integrità poiché all'interno del crittogramma è presente un MAC calcolato sul testo stesso. Un avversario per creare un crittogramma valido dovrebbe possedere sia k_1 che k_2 .

Il secondo protocollo non garantisce integrità perché, sapendo x è possibile creare un nuovo $E_{P_K}(H(x))$.

non-repudiation

Il primo protocollo non la garantisce perché:

- k_1 è una chiave di un cifrario simmetrico e quindi può anche essere usata da altri utenti.
- k_1 è una chiave di un HMAC, quindi simmetrica e quindi può anche essere usata da altri utenti.

Il secondo protocollo non la garantisce perché chiunque può produrre quel crittogramma, dato che x è in chiaro e che è possibile creare un nuovo $E_{P_K}(H(x))$.

Domande orale

Tra parentesi c'è la frequenza.

****Shannon****

1. teorema di Shannon - Cifrario perfetto + dimostrazione (x5)
2. Shannon's definition of perfect cipher (x5)
3. teorema di Shannon, enunciato e dimostrazione (x5)

****Diffie-Hellman, Operazioni su $\mathbb{Z}_p^*$ e Man in the middle problem****

1. Diffie-hellman + perché si lavora in un sottogruppo di $\mathbb{Z}_p^*$ (x4)
2. Protocollo Diffie-Hellman, come funziona? Si descriva il suo funzionamento e come stabilire i parametri necessari (x4)

3. Diffie-hellman (per scambio di chiavi) (x4)

4. Man-in-the-middle Attack in Diffie-hellman (x4)

****Perfect Forward Secrecy, problemi e vantaggi****

1. Perfect forward secrecy (Definizione + esempio di come si può ottenere) (x3)

2. Perfect forward secrecy (x3)

3. problema della perfect forward secrecy? (x3)

****RSA****

1. Sicurezza RSA + perché l'algoritmo base non è sicuro (x6)

2. RSA key key-generation (x6)

3. RSA scolastico (rsa plain): perché non si usa in pratica (x6)

4. algoritmo di generazione delle chiavi in RSA (x6)

5. RSA: algo generazione della chiave (x6)

6. RSA encryption and decryption performance improvements (x6)

****Hash functions: 3-proprietà, Merkle-Damgaard scheme, funzioni di compressione, guessing attack, birthday attack****

1. Proprietà di una funzione Hash + attacchi + Merkle Damgard (Concetto generale + schema se si ricorda) (x7)

2. Funzioni hash (3 proprietà, attacchi alle varie proprietà) (x7)

3. Proprietà funzioni Hash, guessing attack (funzionamento e complessità) (x7)

4. Guessing attack (x7)

5. Birthday attack (paradosso del compleanno) (x7)

6. schema di Merkle-Damgaard (hash più grandi da hash piccoli(?)), Come si realizza la funzione $h(\text{piccolo})$ (x7)

7. Definizione di funzione Hash sicura, quali proprietà deve possedere? Imposta e calcola la probabilità di un Birthday attack, con la complessità. Schema Merkle-Damgaard. (x7)

****Certificati: Informazioni minime, operazioni di verifica****

1. Quali sono le informazioni minime che ci aspettiamo di trovare dentro un certificato? quali sono i controlli da fare prima di utilizzarlo? (x2)

2. Certificates (including OCSP, CRL and Cross certification) (x2)

3. When is a certificate considered valid? (x2)

****Cifrari a blocchi ed Encryption modes: CBC, ECB, OFB e CTR****

1. CBC (x10)

2. ECB (x11)

3. differenze CBC/ECB (x10)

4. CBC and error propagation (x10)

5. Argue whether CBC can be used with public key encryption schemes (x10)

6. schema modalità CBC (x10)

7. Block cipher: per costruire cifrario a caratteri, modalità -> OFB, CFB, CTR; (dimostrazione che OFB non può essere un cifrario perfetto NON SPIEGATO) (x10)

8. Block cipher: costruzione cifrario perfetto, mostrare che non è possibile in pratica (permutazioni...); che permutazione implementa un cifrario? (x10)

9. Da Block Cipher a stream cipher, Encryption modes (x10)

****DES, 2-DES e 3-DES****

1. cifrario DES quando si usa? per quale motivo il 3DES è sicuro e la 2DES no? complessità di storage? complessità sui dati? trascurabile quella sui dati. se usassi 4DES? (x2)

2. DES, 2-DES e 3-DES: quanto conviene? (x2)

****Square and multiply****

1. $a^x \bmod n$ (multiply, generalizzazione) (x1)

2. parametro "e" in RSA (?) come si determina, a^x , square&multiply (esponente deve contenere il minor numero di 1-calcolo combinatorio(?)) (x1)

****MAC e HMAC****

1. MAC: costruzione HMAC (x1)

****DLP sulle curve ellittiche****

1. DLP non generici, $\mathbb{Z}_p^*$, vantaggio di passare alle curve ellittiche? (x1)

****Dictionary Attack e Rainbow-Table Attack****

1. Dictionary attack (è un kpa) (x1)

2. Da cosa dipende la dimensione del dizionario nel dictionary attack (da 2^n) (x1)

****RNG: proprietà per la sicurezza****

1. cifrari a caratteri: approssimazione block cipher, caratteristiche che deve avere un PRG per essere sicuro; LFSR: predicibile (x1)

****Varie****

1. Somma tra int (x1)

2. OS command injection (x1)

3. SQL Injection, quali sono le tecniche per contrastarlo? (x1)

4. Black box (x1)

5. Discuss about authenticated encryption schemes and argue about their security (x1)

6. Kerberos (x1)

7. Kerberos delegation (proxy and forwardable tickets) (x1)

8. Cifratura multipla (2E, 3DES): quanto conviene? (x1)

9. problema della canonizzazione dei nomi nel file system? limite whitelist e blacklist? (x1)

10. spiegare le vulnerabilità legate alle funzione di libreria, scanf e printf che hanno un numero di parametri variabili (x1)